

Introduction to C Programming

A Beginner's Course

STUDENT MANUAL

A complete, offline, printable reference covering the entire course —
Part 1 (Modules 0–9) and Part 2 (Modules 10–19), start to finish.

Table of Contents

Welcome / How This Course Works

Welcome! This course takes you from **never having written a line of code** to **designing your own leak-free, multi-file data structures**, tracked in Git and submitted through GitHub Classroom. No prior programming experience is assumed.

The course comes in two parts:

- **Part 1 — Modules 0–9.** Zero to a working, file-backed C program: syntax, types, control flow, arrays and strings, functions, pointers, structs, and a team capstone.
- **Part 2 — Modules 10–19.** The actual dividing line between "I can write C" and "I've written some C once": recursion, dynamic memory, linked lists and the structures built on them, multi-file program organization, debugging tools, and a second, harder capstone.

Part 2 assumes Part 1 is finished — Module 10 does not re-explain anything from Modules 0–9.

Built to work with or without a lecturer in the room

Every lesson is written so you can teach yourself from it directly — the *why* always comes before the *how*, every claim is backed by code you actually run, and nothing assumes a lecturer will fill in a gap out loud. If you have an instructor, they'll add depth, war stories, and live debugging. If you don't, you can still get every concept from the page.

Every module starts with a hook

Before any theory, every module opens with a **Hook** — a short, genuinely fun demo or puzzle you run *first*, before you understand how it works. Curiosity ("wait, why did that happen?") is the entire teaching strategy — it's a far stronger reason to keep going than "you'll need this later." By the end of each module you'll understand exactly how your own hook worked.

How this course is built

Every module (after Setup) follows the same shape:

1. **README.md** — the lesson. A hook first, then concepts explained in plain language, with common beginner mistakes called out explicitly.
2. **examples/** — small, runnable `.c` files that demonstrate exactly one idea at a time. Compile and run every single one yourself. Reading code you didn't type teaches you far less than watching your own build succeed (or fail) at the terminal.
3. **exercises/** — practice problems with starter files containing `TODO` markers. A `solutions/` folder sits next to them — try for at least 15 minutes before you look.
4. **project/** — a small, fun program that only uses what you've learned *so far*. This is the payoff for the module — something you'd actually want to show a friend.

Roadmap

Part 1

#	Module	You will be able to...
0	Setup: Tools, Git & GitHub	Install a compiler, use VS Code,

#	Module	You will be able to...
	Classroom	and submit work through GitHub Classroom
1	Basics of Programming	Explain how a C program becomes a running program, and write/compile your first one
2	Data Types & Operators	Choose the right type for a piece of data and compute with it correctly
3	Variables & Input/Output	Store data, name it well, and write programs that talk back to the user
4	Control Structures	Make decisions (<code>if/switch</code>) and repeat work (loops)
5	Arrays & Strings	Store and process collections of values and text
6	Functions	Break a program into reusable, named, testable pieces
7	Pointers	Work directly with memory addresses — the idea that makes C, C
8	Structs & File I/O	Design your own data records and make programs remember things after they close
9	Capstone Project	Combine everything into one real program, built in a team and submitted via GitHub Classroom

Part 2

#	Module	You will be able to...
10	Recursion	Write a function that calls itself correctly, and trace its call stack by hand
11	Dynamic Memory Allocation	Use <code>malloc/calloc/realloc/free</code> and stop being limited to fixed-size arrays
12	Linked Lists	Build a growable data

#	Module	You will be able to...
		structure from structs, pointers, and dynamic memory
13	Stacks & Queues	Implement LIFO/FIFO structures and recognize where each applies
14	Multi-File Programs	Split a program across <code>.h/.c</code> files with include guards, and compile them together
15	Debugging & Memory Safety	Use <code>gdb</code> and <code>valgrind</code> to find a crash's real cause and catch memory leaks
16	Enums, Unions & Bitwise Operators	Model fixed states cleanly and manipulate individual bits
17	Command-Line Arguments & CLI Tools	Read <code>argc/argv</code> and build a real terminal tool, not just a menu loop
18	Sorting & Searching Algorithms	Implement classic sorts and searches, with an intuition for why some are faster
19	Part 2 Capstone Project	Build a leak-free, multi-file program on a dynamic data structure, submitted via GitHub Classroom

Ground rules for how we'll work

- **Type the code yourself.** Copy-pasting defeats the purpose — your fingers and your mistakes are how the syntax sticks.
- **Read every compiler error fully,** top to bottom, before asking for help. The first error in a long list is usually the real one; everything after it is often noise caused by that first mistake.
- **Compile often.** Write 3–5 lines, then compile. Don't write 100 lines and try to debug them all at once.
- **Commit often.** After finishing an example, exercise, or project step, commit it. Module 0 shows you exactly how. Small, frequent commits are a professional habit you're building from day one — and they're also your evidence of steady progress once the capstone is a team project.

How work gets submitted

Every exercise, module project, and both capstones are submitted through **GitHub Classroom**, not by email or file upload:

- Individual modules (0–8, 10–18): you accept a per-student assignment link once (Module 0 walks through this), which gives you your own private repo. Commit and push your work there as you go — the last commit before a deadline is what gets graded.
- The capstones (Module 9, Module 19): each is a **group assignment** — GitHub Classroom creates one shared repo per team. See **Module 9** and **Module 19** for exactly how teams are formed and how many people are on yours.

Prerequisites

None, other than curiosity and a willingness to make (and fix) mistakes. Mistakes are not a sign you're bad at this — they're the main way anyone learns to program. Start with **Module 0: Setup**.

Part 1 — Foundations (Modules 0–9)

Zero to a working, file-backed C program: syntax, types, control flow, arrays and strings, functions, pointers, structs, and a team capstone.

Module 0 — Setup: Tools, Git & GitHub Classroom

Hook: your first build, before you understand a single line

Before anything else, let's prove the machinery works. Create a file called `hook.c` anywhere on your computer and paste this in exactly:

```
#include <stdio.h>

int main(void) {
    int year = 2026;
    printf("If you started today, by %d you could have written your\n", year);
    printf("own game, your own tool, or your own excuse for why the\n");
    printf("bug definitely isn't your fault. Let's find out.\n");
    return 0;
}
```

Once your compiler is installed below, run:

```
gcc hook.c -o hook
./hook
```

You just turned text into a running program on your own machine. You don't understand `#include`, `printf`, or `return` yet — that's fine, that's the entire rest of this course. Keep that feeling of "it worked and I don't fully know why" in mind; every module from here replaces one piece of "I don't know why" with "I know exactly why."

1. Install a C compiler

C needs to be **compiled** — translated into machine code before it runs (more on this in Module 1). You need a compiler on your machine.

macOS

```
xcode-select --install
```

This installs Apple's Command Line Tools, which includes `clang` (a C compiler). This course uses `gcc` as the command name throughout — on macOS, `gcc` is typically aliased to `clang` already. Check with:

```
gcc --version
```

Windows

Install **WSL (Windows Subsystem for Linux)** — open PowerShell as Administrator and run `wsl --install`, then restart. Inside the Ubuntu terminal that opens, run:

```
sudo apt update && sudo apt install build-essential -y
```

This gives you `gcc`, `make`, and a real Linux terminal — the same environment your instructor and classmates are using, which avoids "works on my machine" problems.

Linux

```
sudo apt install build-essential -y      # Debian/Ubuntu
sudo dnf groupinstall "Development Tools" # Fedora
```

Verify on any platform:

```
gcc --version
```

You should see a version number, not "command not found."

2. Install VS Code

Download from code.visualstudio.com. Then install two extensions from the Extensions panel (Cmd/Ctrl+Shift+X):

- **C/C++** (by Microsoft) — syntax highlighting and IntelliSense.
- **Code Runner** (by Jun Han) — lets you run a `.c` file with one click.

3. Install and configure Git

Git is almost certainly already installed (`git --version` to check). If not, install it from git-scm.com. Then tell Git who you are — this name and email get attached to every commit you make:

```
git config --global user.name "Your Full Name"
git config --global user.email "the-email-you-use-for-github@example.com"
```

4. Create a GitHub account

If you don't already have one, sign up at github.com. Use a professional-looking username — this account will hold your work long after this course ends, and employers do look.

5. Accept your first GitHub Classroom assignment

This course does **not** collect work by email or file upload — everything goes through **GitHub Classroom**.

1. Your instructor shares an **assignment invite link** for Module 0 (in a codespace, LMS post, or class chat).
2. Open the link and sign in with your GitHub account.
3. Click **Accept this assignment**. GitHub Classroom creates a private repository just for you (something like `module-0-setup-yourusername`), copied from a starter template.
4. Wait a few seconds for "Your assignment repository has been created" and click through to it.

6. Clone your repository and make your first commit

On your computer, in a terminal, navigate to where you keep your code and clone the repo GitHub Classroom just created for you:

```
git clone https://github.com/YOUR-CLASSROOM-ORG/module-0-setup-
yourusername.git
cd module-0-setup-yourusername
```

Open the folder in VS Code (`code .` from that terminal, or File → Open Folder). Add a `hook.c` file with the hook program from the top of this page (personalize the message if you like), then:

```
git add hook.c
git commit -m "Complete Module 0 setup"
git push
```

Refresh the repository page on GitHub — you should see your file and your commit. **This is your workflow for every module from now on:** edit code locally, `git add` the files you changed, `git commit` with a short description, `git push` to send it to your GitHub Classroom repo. Nothing is submitted until it's pushed.

Common setup problems

- `gcc: command not found` — the compiler isn't installed, or your terminal was opened before installation finished. Close and reopen the terminal.
- `git push` asks for a password and rejects it — GitHub no longer accepts account passwords for this. Use a **personal access token** as the password, or set up SSH keys (ask your instructor which your class uses).
- Nothing shows up on GitHub after `git push` — check `git status` first; if it says "nothing to commit," you likely forgot `git add` before `git commit`.

Try it yourself

1. Confirm `gcc --version` and `git --version` both work.
2. Accept the Module 0 GitHub Classroom assignment.
3. Clone it, add your `hook.c`, commit, and push.
4. Confirm your commit appears on GitHub.

Next: **Module 1 — Basics of Programming.**

Module 1 — Basics of Programming

Hook: the world's smallest fortune teller

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>

int main(void) {
    printf("You will encounter a strange creature today.\n");
    printf("It has six legs, compound eyes, and excellent opinions.\n");
    printf("Its name is: ");
    printf("your cat.\n");
    return 0;
}

gcc fortune.c -o fortune
./fortune
```

Four `printf` calls, one program, one joke that lands because of the line break placement. By the end of this module you'll understand exactly why that third `printf` doesn't start a new line, and you'll build your own version of this in the module project.

What is a program, really?

A computer only understands one language: electrical signals interpreted as numbers — **machine code**. Nobody writes programs directly in machine code anymore. Instead, we write in a **language humans can read** (like C), and a program called a **compiler** translates it into machine code for us.

you write		compiler translates		computer runs
hello.c	--->	(gcc)	--->	hello
(C source)				(machine code)

This is exactly what you did in Module 0 when you ran `gcc hook.c -o hook`. Every C program you ever write follows this same pipeline: **source code** → **compiler** → **executable** → **run**.

C is a **compiled language**. Contrast this with languages like Python, which are **interpreted** — run line-by-line by another program every time, with no separate "build" step. This is why compiled programs tend to run faster (all the translation work happens once, ahead of time) but have an extra step (you must recompile after every change).

Anatomy of a C program

Look at the smallest complete, useful C program:

```
#include <stdio.h>

int main(void) {
    printf("Hello, C!\n");
    return 0;
}
```

Let's take it apart line by line.

`#include <stdio.h>`

C's core language is small — even printing to the screen isn't built in. Instead, functionality lives in **libraries** you pull in with `#include`. `stdio.h` ("standard input/output header") gives you `printf`

(for printing) and `scanf` (for reading input, coming in Module 3). This line must come before you use anything from that library.

```
int main(void) { ... }
```

Every C program has exactly one `main` function — it's the **entry point**: the first code that runs when your program starts. `int` means `main` reports back a whole number when it finishes (see `return 0` below). `void` between the parentheses means "this version of `main` takes no inputs" — you'll see a different form of `main` if you ever handle command-line arguments, but this is the one you'll use for now. The `{` and `}` mark where `main`'s body begins and ends — everything a function does lives between its braces.

```
printf("Hello, C!\n");
```

- `printf` — "print formatted" — a function from `stdio.h` that writes text to the screen.
- `"Hello, C!\n"` — a **string literal**: literal text, always in double quotes. `\n` is a single special character meaning "newline" — it's what moves output to the next line. Without it, the next `printf` would continue on the *same* line (exactly what the hook program exploited).
- `;` — a **semicolon** ends every statement in C. Forget one and the compiler will complain — usually about the *next* line, which confuses beginners. If an error message makes no sense, check the line *above* it for a missing semicolon first.

```
return 0;
```

Tells whoever ran the program "I finished successfully." `0` conventionally means success; any other number signals some kind of error occurred. You won't use this deliberately much yet, but it must be there in `main`.

Statements, whitespace, and comments

- A **statement** is one instruction, ending in `;`. C doesn't care about line breaks or indentation the way Python does — they're purely for humans to read the code. *Always* indent consistently anyway; VS Code's C/C++ extension does this for you automatically.
- **Comments** are notes for humans that the compiler ignores entirely:

```
// A single-line comment — everything after // on this line is ignored.
```

```
/* A multi-line comment.
```

```
    Everything between /* and */ is ignored, across as many lines as you
like. */
```

Use comments to explain *why* something is done a non-obvious way — not to restate what the code already says.

Compiling and running — the two ways you'll use in this course

Terminal (do this to really understand it):

```
gcc myfile.c -o myfile
./myfile
```

VS Code Code Runner (fast iteration once you're comfortable): open the `.c` file and click the  button in the top-right corner, or press `Ctrl+Alt+N`.

Both do the same thing. Use the terminal version whenever you want to be sure you understand exactly what's happening — it's also what you'll use on any computer that doesn't have VS Code.

Errors are normal — read them, don't fear them

Try compiling this broken version (missing semicolon):

```
#include <stdio.h>

int main(void) {
    printf("Oops\n")
    return 0;
}
```

gcc will report something like: error: expected ';' before 'return'

That's the compiler pointing almost exactly at the mistake. Reading compiler errors is a skill — you'll get fast at it. The habit that matters most right now: **fix the first error first, then recompile**. One missing semicolon can produce five confusing-looking errors below it; they usually all disappear once you fix the real one.

Common beginner mistakes

- Forgetting the semicolon at the end of a statement.
- Mismatched { and } — VS Code highlights the matching brace when your cursor is next to one; use it.
- Forgetting `#include <stdio.h>` and then using `printf` — some compilers will still let this slide with a warning, but always include it; never rely on it being implicit.
- Forgetting `\n` and being confused why two `printf` calls' output ran together on one line.
- Typos in `printf` (e.g. `Printf`, `PRINTF`) — C is **case-sensitive**.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one.
2. Complete **exercises/exercise1.c**.
3. Build the **module project**.
4. Commit and push your work (see Module 0 §6 if you need a refresher):

```
git add .
git commit -m "Complete Module 1: basics of programming"
git push
```

Next: **Module 2 — Data Types & Operators**.

Module 2 — Data Types & Operators

Hook: why did math just break?

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>

int main(void) {
    printf("Splitting 7 slices of pizza among 2 people: %d slices each\n", 7 /
2);

    unsigned char tinyBox = 255;
    tinyBox = tinyBox + 1;
    printf("One more than 255 in a box that only holds 0-255: %d\n", tinyBox);

    return 0;
}
gcc hook.c -o hook
./hook
```

You should see 3 slices each (not 3.5 — did someone eat half a slice?), and then 0 where you'd expect 256. Neither of these is a bug in your program. Both are C doing *exactly* what it was told, using the rules for **data types** and **operators** you're about to learn. By the end of this module you'll know precisely why 7 divided by 2 lost its remainder, and why a number can "wrap around" back to zero — and you'll use both facts on purpose in this module's project.

Why data types exist

Every piece of data in a C program lives in memory as a fixed-size box of bits. A **data type** tells the compiler two things: how big that box is, and how to interpret the bits inside it. Get the type wrong for what you're storing, and you get wrong (but silent!) answers — like the hook just showed you. This is different from what you're used to if you've ever used a calculator or spreadsheet, where numbers "just work." In C, you choose the box size up front.

The basic types

```
#include <stdio.h>

int main(void) {
    int wholeNumber = 42;
    float singlePrecision = 3.14f;
    double doublePrecision = 3.14159265358979;
    char letter = 'A';

    printf("%d\n", wholeNumber);
    printf("%f\n", singlePrecision);
    printf("%f\n", doublePrecision);
    printf("%c\n", letter);

    return 0;
}
```

- **int** — whole numbers, positive or negative, no decimal point: 42, -7, 0.

- **float** — numbers with a decimal point, stored with limited precision (about 6-7 reliable digits). Written with an `f` suffix in code, e.g. `3.14f`.
- **double** — also decimal numbers, but roughly twice the storage of `float` and about twice the precision. This is C's *default* choice for decimal numbers when you're not tight on memory — prefer it over `float` unless you have a specific reason not to.
- **char** — a single character, written in **single** quotes: `'A'`, `'7'`, `'$'`. Under the hood it's just a small integer (each character has a numeric code), which is why arithmetic on `char` values is legal, if unusual, in C.

You'll also see `short` and `long` mentioned in other people's code — they're `int` variants with smaller or larger ranges. As a beginner you'll rarely need them; stick to `int`, `float`, `double`, and `char` for now.

This line is new too: `int wholeNumber = 42;` **declares** a variable (reserves a labeled box of memory of that type) and **initializes** it (puts a value in the box) in one step. We're using variables here purely as a way to hold values so we can inspect types and operators — Module 3 goes much deeper on naming them well, `const`, and reading values in from the user with `scanf`.

Picking the right format specifier

`printf` needs to be told what type you're handing it, using a **format specifier** inside the string:

Type	Specifier
<code>int</code>	<code>%d</code>
<code>float/double</code>	<code>%f</code>
<code>char</code>	<code>%c</code>

Get this wrong (e.g. `%d` for a `float`) and you'll get garbage output — the compiler often won't stop you, so double-check it yourself. There's also `%s` for printing strings of text, but strings are a Module 5 topic — you won't need it yet.

sizeof — how big is that box, really?

`sizeof` tells you exactly how many bytes a type (or a variable) occupies in memory:

```
printf("%zu\n", sizeof(int));
printf("%zu\n", sizeof(wholeNumber));
```

Note the format specifier: `sizeof` produces a special unsigned type, and `%zu` is the correct specifier for it — not `%d`. On a typical machine you'll see `int` and `float` at 4 bytes, `double` at 8 bytes, and `char` at exactly 1 byte (by definition — `char` is always 1 byte in C). This is also the mechanism behind the hook: `unsigned char` only has 1 byte to work with, giving it 256 possible values (0-255). Add 1 past the top of its range and there's nowhere for the extra bit to go — it silently **wraps around** back to 0.

Arithmetic operators — and the integer division trap

C gives you the five arithmetic operators you'd expect: `+` `-` `*` `/` `%`. The first three behave the way you'd guess. The last two need care.

```
int apples = 7;
int friends = 2;

printf("%d\n", apples / friends);    // 3 -- NOT 3.5
printf("%d\n", apples % friends);    // 1 -- the remainder
```

When both operands of / are integers, C performs integer division: the decimal portion is discarded entirely, not rounded. $7 / 2$ is 3, full stop. This is exactly what the hook demonstrated, and it is *probably the single most common source of silently-wrong output* for beginners in C. To get a true decimal answer, at least one operand must be a `float` or `double` — which brings us to conversion and casting.

`%` (modulo) gives you the remainder left over from integer division: $7 \% 2$ is 1, because 7 divided by 2 is 3 remainder 1. It only works on integer types.

Type conversion and casting

Implicit conversion happens automatically when an expression mixes types — C "promotes" the smaller/less-precise type up to match:

```
int wholeScore = 7;
float bonus = 2.5f;
float total = wholeScore + bonus;    // wholeScore is promoted to float first
```

This is convenient, but it doesn't save you from the integer division trap above — that trap only happens *before* any promotion, when **both** operands are already integers. To fix it, you need an **explicit cast**: write the target type in parentheses right before the value you want converted.

```
int a = 7;
int b = 2;

printf("%d\n", a / b);                // 3 -- integer division
printf("%f\n", (float)a / b);         // 3.500000 -- a is converted to float FIRST
```

`(float)a` converts just `a` to a `float` *before* the division runs. Since one operand is now a `float`, the whole division becomes float division. This one technique — casting one operand before dividing — is the fix for the hook's first surprise, and you'll use it constantly.

Relational and logical operators

Relational operators compare two values and produce 1 (true) or 0 (false) — C has no separate boolean type for this:

```
int age = 20;

printf("%d\n", age == 20);    // 1 (equal to)
printf("%d\n", age != 20);    // 0 (not equal to)
printf("%d\n", age > 18);     // 1 (greater than)
printf("%d\n", age < 18);     // 0 (less than)
printf("%d\n", age >= 20);    // 1 (greater than or equal to)
printf("%d\n", age <= 19);    // 0 (less than or equal to)
```

Watch out: `==` (comparison) and `=` (assignment) are different operators that look almost identical — mixing them up is a classic bug you'll meet again in Module 4.

Logical operators combine 0/1 results: `&&` (and — true only if both sides are true), `||` (or — true if either side is true), and `!` (not — flips true to false and vice versa):

```
printf("%d\n", (age > 18) && (age < 65)); // 1 -- both true
printf("%d\n", (age < 18) || (age > 65)); // 0 -- both false
printf("%d\n", !(age == 20));           // 0 -- flips 1 to 0
```

Right now we're just printing what these operators produce. Using them to actually branch your program's behavior (`if` statements) is Module 4's job — for now, just get comfortable with the fact that a comparison *is* a value in C: 0 or 1, ready to be printed, stored, or combined with others.

Operator precedence

Just like in math class, `*` and `/` run before `+` and `-`:

```
printf("%d\n", 2 + 3 * 4); // 14, not 20
printf("%d\n", (2 + 3) * 4); // 20 -- parentheses run first
```

You don't need to memorize a full precedence table right now. The practical rule: **when you're not 100% sure how an expression will be evaluated, add parentheses.** It costs nothing, makes your intent obvious to anyone reading the code, and removes all doubt.

Common beginner mistakes

- Assuming `int / int` gives a decimal answer. It doesn't — `7 / 2` is 3. Cast at least one operand to `float` or `double` if you need a real decimal result.
- Using the wrong `printf` format specifier for a type (e.g. `%d` for a `float`) — this often doesn't error, it just prints garbage.
- Using `%d` for the result of `sizeof` instead of `%zu`.
- Confusing `=` (assignment) with `==` (comparison).
- Forgetting single quotes for `char` literals (`'A'`, not `"A"` — double quotes are for strings, which are a different, later topic).
- Casting the *wrong* operand or casting too late — `(float)(a / b)` still does integer division first and only converts the already-wrong answer; you must cast **before** the division happens, e.g. `(float)a / b`.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one. Predict the output before you run it, then check.
2. Complete **exercises/exercisel.c**.
3. Build the **module project**.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 2: data types and operators"
git push
```

Next: **Module 3 — Variables & Input/Output.**

Module 3 — Variables & Input/Output

Hook: the instant mind reader

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>

int main(void) {
    int age;

    printf("Think of your age. Don't tell anyone. Just type it in: ");
    scanf("%d", &age);

    int birth_year_guess = 2026 - age;
    int days_alive_guess = age * 365;

    printf("\nI sense... you were born around the year %d.\n",
    birth_year_guess);
    printf("You have been alive for roughly %d days.\n", days_alive_guess);
    printf("I am basically a wizard.\n");

    return 0;
}
gcc mindreader.c -o mindreader
./mindreader
```

Type in your age and watch it "read your mind." There's no trick — just a variable holding the number you typed, and two lines of arithmetic. By the end of this module you'll understand every piece of this program: how `age` stores what you typed, what that `&` before it is doing, and how to build interactive programs like this from scratch. You'll build a bigger version of it in the module project.

What is a variable, really?

Every program so far has only ever printed text that was baked into the code itself. That's not very useful — a real program needs to work with information that changes: a score, a price, an age typed in by whoever is running it. A **variable** is a named box in the computer's memory that holds a value your program can read and change while it runs.

```
int age = 16;
^   ^   ^
|   |   +-- the value stored in the box right now
|   +-- the name you use to refer to the box
+-- the type of value the box is allowed to hold (Module 2)
```

You've already used types (`int`, `float`, `double`, `char`) and literal values in Module 2. A variable is what lets a value have a *name* you can reuse, print, and update throughout a program instead of typing the literal value over and over.

Naming variables: rules and conventions

C enforces a small set of **rules** for names (the compiler will reject anything that breaks them):

- Names can contain letters, digits, and underscores (`_`) only.
- A name **cannot start with a digit** (`1st_place` is illegal; `first_place` is fine).

- C is **case-sensitive** — `age`, `Age`, and `AGE` are three different names.
- You cannot use a **reserved word** as a name — words the language already uses for something else, like `int`, `float`, `return`, or `const` (more on `const` below).

Beyond what the compiler requires, good C code follows **conventions** — habits the whole C community shares so anyone's code is easy to read:

- Use **meaningful names**. `s` tells you nothing; `average_score` tells you exactly what's stored there. You are writing code for the next human who reads it — often that human is you, in six months.
- Use **snake_case** for variable names: all lowercase, words joined by underscores (`number_of_students`, not `numberOfStudents` or `NumberOfStudents`). This is the idiomatic style in C, even though other languages prefer different conventions.

Declaring, initializing, and re-assigning

Declaring a variable creates the box but doesn't necessarily put anything meaningful in it yet:

```
int score;
```

Initializing means giving it a value at the moment you declare it — almost always the better habit:

```
int score = 0;
```

You can also declare first and assign a value on a later line using `=`:

```
int score;
score = 0;
```

Once a variable already exists, using `=` again doesn't declare a new variable — it **re-assigns** (overwrites) the value already stored there:

```
int age = 16;
age = age + 1; // reads the current value (16), adds 1, stores 17 back
```

Notice `age = age + 1;` is not an equation in the mathematical sense — `=` in C means "compute the right side, then store the result in the variable on the left," not "these two things are equal."

Why uninitialized variables are dangerous

Declaring a variable **without** initializing it leaves whatever bits happened to already be sitting at that spot in memory — a **garbage value**. It is not guaranteed to be `0`, or any predictable value at all:

```
int mystery;
printf("%d\n", mystery); // could print anything -- don't do this
```

This is a classic source of bugs that are maddening to track down, because the program might *seem* to work (garbage value happens to be harmless) until it doesn't (garbage value happens to be huge, negative, or otherwise breaks your logic) on a different run or a different computer. The habit that avoids this entirely: **always initialize a variable when you declare it**, unless you are about to immediately read a value into it (as you'll do with `scanf` below).

`const`: values that should never change

Some values in a program should be fixed for its entire run — a mathematical constant, a maximum class size, a fixed price. Marking a variable `const` tells the compiler to enforce that:

```
const double PI = 3.14159;
const int MAX_STUDENTS = 30;
```

Try to assign a new value to a `const` variable anywhere later in the program, and `gcc` will refuse to compile it. That's a feature, not an annoyance — it turns "a value silently got changed somewhere it shouldn't have" from a confusing runtime bug into an immediate, obvious compiler error. Use `const` for any variable you can already tell should never change after it's set.

Reading user input with `scanf`

Every program until now has been a one-way conversation — it talks, you listen. `scanf` (from `stdio.h`, the same header that gives you `printf`) lets your program **read** input the user types.

```
int age;
printf("Enter your age: ");
scanf("%d", &age);
```

A few things to notice:

- The **format specifier** works the same way it did for `printf` in Module 2 — `%d` for `int`, `%f` for `float`, `%c` for a single `char`.
- The **& before the variable name** is required. It means "the address of this variable" — it tells `scanf` exactly *where* in memory to store the value it reads, rather than handing it a copy of whatever's currently there. You'll fully understand why this works the way it does in Module 7 (Pointers). For now, the rule is simple: **always write & before the variable name in `scanf`** (for the types you know so far — this changes once you meet strings and arrays in Module 5).
- Always `printf` a prompt *before* calling `scanf`, so the user knows the program is waiting for them to type something.

Reading a `float` works the same way:

```
float price;
printf("Enter a price: ");
scanf("%f", &price);
```

`scanf` with `char` — and the leading-space bug

Reading a single character looks like it should be just as simple:

```
char grade;
scanf("%c", &grade);
```

But there's a very common trap. Say your program first reads an `int` with `scanf("%d", &score)`, then immediately tries to read a `char`:

```
int score;
char grade;

scanf("%d", &score);
scanf("%c", &grade); // BUG: this often doesn't wait for you to type
anything
```

When you type a number and press Enter, `scanf("%d", ...)` reads the digits but leaves the newline character (`\n`) from pressing Enter sitting in the input buffer, untouched. The very next `scanf("%c", ...)` immediately reads *that leftover newline* as if it were your answer — your program appears to skip right past asking for the character.

The fix is a single space, written before `%c`:

```
scanf(" %c", &grade);
```

That leading space tells `scanf`: "skip over any whitespace (spaces, tabs, or leftover newlines) sitting in the buffer first, then read the next real character." This one-character fix resolves one of the most common and confusing beginner bugs with `scanf` — if a `%c` read seems to get silently skipped, this is almost always why.

Putting it together: read, compute, print

Every interactive program you'll write follows the same three-step shape: **read input, compute something with it, print a result.**

```
float num1, num2;

printf("Enter the first number: ");
scanf("%f", &num1);

printf("Enter the second number: ");
scanf("%f", &num2);

float sum = num1 + num2;
printf("%.2f + %.2f = %.2f\n", num1, num2, sum);
```

This is exactly the pattern behind the mind-reader hook at the top of this module — read an age, compute a couple of values from it with arithmetic, print sentences that reference the real numbers the user typed. That's what makes it feel personal instead of generic.

Common beginner mistakes

- Forgetting `&` before a variable name in `scanf` — the program will usually crash or behave unpredictably.
- Forgetting the leading space before `%c` in `scanf(" %c", ...)` when a numeric read came before it.
- Using the wrong format specifier for the variable's type (e.g. `scanf("%d", &price)` where `price` is a `float`) — this silently reads garbage instead of warning you.
- Declaring a variable without initializing it and then reading its value before ever assigning one.
- Trying to re-assign a `const` variable after it's been set.
- Using non-descriptive names (`x`, `a`, `temp1`) instead of names that say what the value actually represents.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one. Try feeding the `scanf` examples different input values.
2. Complete **exercises/exercisel.c**.
3. Build the **module project**.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 3: variables and input/output"
git push
```

Next: **Module 4 — Control Structures.**

Module 4 — Control Structures

Hook: Rock, Paper, Scissors vs. the computer

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    srand(time(NULL));

    int playerChoice;
    printf("Rock (0), Paper (1), or Scissors (2)? ");
    scanf("%d", &playerChoice);

    int computerChoice = rand() % 3;

    printf("You chose:      ");
    if (playerChoice == 0) {
        printf("Rock\n");
    } else if (playerChoice == 1) {
        printf("Paper\n");
    } else {
        printf("Scissors\n");
    }

    printf("Computer chose: ");
    if (computerChoice == 0) {
        printf("Rock\n");
    } else if (computerChoice == 1) {
        printf("Paper\n");
    } else {
        printf("Scissors\n");
    }

    if (playerChoice == computerChoice) {
        printf("It's a tie!\n");
    } else if ((playerChoice == 0 && computerChoice == 2) ||
               (playerChoice == 1 && computerChoice == 0) ||
               (playerChoice == 2 && computerChoice == 1)) {
        printf("You win!\n");
    } else {
        printf("Computer wins!\n");
    }

    return 0;
}
gcc rps.c -o rps
./rps
```

Run it a few times and try all three choices. Two things in there are new: `if/else if/else`, which decide which lines of code actually run, and `rand()`, which gets you a random number. You've already met the building blocks these depend on — Module 2's `==`, `&&`, and `||` are exactly what's deciding the

winner above. By the end of this module you'll understand every line of this, and you'll build a full Number Guessing Game using the same ideas in this module's project.

Why control structures?

Every program you've written so far runs top to bottom, once, no exceptions — line 1, then line 2, then line 3, done. That's fine for a fortune teller, but real programs need to **make decisions** ("if the guess is too high, say so") and **repeat work** ("keep asking until the player gets it right"). `if`, `switch`, `while`, `do-while`, and `for` are how C does both. They're called **control structures** because they control which statements run, and how many times.

if / else if / else

An `if` runs a block of code only when its condition is true. Recall from Module 2 that relational operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) and logical operators (`&&`, `||`, `!`) produce 1 (true) or 0 (false) — that value is exactly what `if` checks.

```
int temperature = 28;

if (temperature >= 30) {
    printf("It's hot outside.\n");
} else if (temperature >= 20) {
    printf("It's a mild day.\n");
} else {
    printf("It's cold.\n");
}
```

C checks each condition **top to bottom** and runs the first block whose condition is true — then it skips every other branch, even if a later condition would also be true. `else if` lets you chain as many conditions as you need; the final `else` (optional) catches everything that didn't match any condition above it. Combine conditions with `&&` ("and", both must be true) and `||` ("or", at least one must be true) — see [examples/01_if_else.c](#).

switch statements

When you're comparing **one variable** against several **exact values**, a `switch` is often cleaner than a long `if/else if` chain:

```
int dayNumber = 3;

switch (dayNumber) {
    case 1:
        printf("Monday\n");
        break;
    case 2:
        printf("Tuesday\n");
        break;
    case 3:
        printf("Wednesday\n");
        break;
    default:
        printf("Not a valid day number.\n");
        break;
}
```

`switch` jumps straight to the matching case and starts running code from there. `break` is what makes it **stop** and jump out of the switch — without it, execution keeps going into the next case too, whether it matches or not. This is called **falling through**, and forgetting `break` is one of the most common beginner switch bugs. See [examples/07_switch_fallthrough_demo.c](#) to watch it happen, then fix it. Always include a default case too — it's your safety net for values that don't match anything, the same job a final `else` does for `if`.

Note: `switch` can only compare against exact constant values — it can't do ranges or `&&/||` conditions. For those, you still need `if/else`.

while loops

A `while` loop repeats its body for as long as its condition stays true. The condition is checked **before** every run — including the first — so if it starts out false, the body never runs at all.

```
int countdown = 5;
while (countdown > 0) {
    printf("%d...\n", countdown);
    countdown = countdown - 1;
}
printf("Liftoff!\n");
```

Use a `while` loop when you're repeating **until something happens**, and you don't know in advance how many times that will take — like asking the player to guess a number until they get it right.

do-while loops

`do-while` is almost the same, except it checks its condition **after** running the body — so the body always runs **at least once**, no matter what the condition is:

```
int attempts = 10;
do {
    printf("This runs even though attempts is already 10.\n");
    attempts = attempts + 1;
} while (attempts < 3);
```

This matters anywhere you need to do something once before you can even check the condition — like asking for a player's first guess before you have anything to compare it to. See [examples/03_while_and_do_while.c](#) for a side-by-side comparison.

for loops

A `for` loop packs three things onto one line, separated by `;`:

```
for (int i = 1; i <= 10; i = i + 1) {
    printf("%d\n", i);
}
```

1. **initialization** — `int i = 1;` runs once, before the loop starts.
2. **condition** — `i <= 10` is checked before every run; the loop stops the moment it's false.
3. **increment** — `i = i + 1` runs after every run of the body.

Use a `for` loop when you know **in advance** how many times you want to repeat something (print a multiplication table, roll a die 5 times). Use a `while` loop instead when the number of repetitions depends on something that happens *during* the loop. Anything you can write as a `for` loop, you could also write as a `while` loop with the initialization before it and the increment as the last line of the body — `for` just keeps those three pieces together so they're harder to forget.

break and continue

Both work inside any loop (`while`, `do-while`, or `for`):

- **break** immediately exits the loop entirely — no more checks, no more iterations.
- **continue** skips the *rest* of the current iteration only, then jumps straight to the next check of the condition — the loop keeps going.

```
for (int i = 1; i <= 15; i = i + 1) {
    if (i % 3 == 0) {
        continue; // skip multiples of 3, keep looping
    }
    if (i > 10) {
        break; // stop the loop completely once i passes 10
    }
    printf("%d\n", i);
}
```

Nesting control structures

Control structures can live inside each other — most commonly, an `if` inside a loop, to check something about each value as the loop produces it:

```
for (int i = 1; i <= 10; i = i + 1) {
    if (i % 2 == 0) {
        printf("%d is even\n", i);
    } else {
        printf("%d is odd\n", i);
    }
}
```

Each nested level is usually indented one more level than the one around it — that indentation is for humans (C ignores whitespace, as you saw in Module 1), but it's what makes nested code readable at a glance.

Random numbers with `rand()` and `srand()`

Computers can't generate truly random numbers — `rand()` (from `<stdlib.h>`) actually produces a long, fixed sequence of numbers that just *looks* random. That's good enough for games and simple programs like ours.

`srand()` "seeds" that sequence — without calling it, `rand()` gives you the exact same numbers every single time you run the program. Seeding with the current time, `srand(time(NULL))` (`time()` comes from `<time.h>`), makes the seed different each run, so the numbers actually change:

```
#include <stdlib.h>
#include <time.h>

srand(time(NULL)); // call this once, near the start of main
int roll = rand() % 6 + 1; // a number from 1 to 6
```

`rand()` alone returns a number between 0 and a very large maximum. `% 6` shrinks it down to 0-5, and `+ 1` shifts it up to 1-6. The general pattern for a random number between `min` and `max` (inclusive) is:

```
rand() % (max - min + 1) + min
```

See `examples/06_random_numbers.c`.

Common beginner mistakes

- Using `=` (assignment) instead of `==` (comparison) inside an `if` condition — `if (x = 5)` compiles, assigns 5 to `x`, and is always true, which is almost never what you meant.
- Forgetting `break` in a `switch` case, causing it to fall through into the next case.
- Writing a `while` loop whose condition never becomes false because you forgot to update the variable it depends on — an **infinite loop**. If your program seems frozen, this is almost always why (Ctrl+C stops it).
- Off-by-one errors in a `for` loop condition — `i < 10` runs 10 times (0-9), `i <= 10` runs 11 times (0-10). Decide which one you actually want and double check it.
- Calling `srand()` more than once, or inside a loop — call it **once**, near the top of `main`, before any calls to `rand()`.
- Confusing `break` and `continue` — `break` leaves the loop for good; `continue` just skips to the next round of it.

Try it yourself

1. Work through every file in `examples/`, compiling and running each one — including the intentionally broken `07_switch_fallthrough_demo.c`, so you see the bug before fixing it.
2. Complete `exercises/exercisel.c`.
3. Build the **module project**.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 4: control structures"
git push
```

Next: **Module 5 — Arrays & Strings.**

Module 5 — Arrays & Strings

Hook: crack the secret message

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char secret[] = "Duudbv krog pdqb ydoxhv lq d urz!";
    int shift = 3;
    int len = strlen(secret);

    for (int i = 0; i < len; i++) {
        char c = secret[i];
        if (c >= 'a' && c <= 'z') {
            secret[i] = 'a' + (c - 'a' - shift + 26) % 26;
        } else if (c >= 'A' && c <= 'Z') {
            secret[i] = 'A' + (c - 'A' - shift + 26) % 26;
        }
        // anything that isn't a letter (spaces, punctuation) is left alone
    }

    printf("Decoding secret message...\n");
    printf("%s\n", secret);

    return 0;
}
gcc hook.c -o hook
./hook
```

Gibberish goes in, an English sentence comes out. `secret` isn't a single value like the `ints` and `chars` you've used so far — it's a whole row of characters that the loop walks across one at a time, shifting each letter three places back through the alphabet. By the end of this module you'll understand exactly how `secret[i]` works, why `char secret[]` can hold a whole sentence, and you'll build your own encoder/decoder in the module project.

What is an array?

Every variable you've used so far holds exactly one value: one `int`, one `float`, one `char`. But plenty of real problems need a *bunch* of related values — five quiz scores, thirty students' ages, a whole word. Declaring 30 separate variables (`score1`, `score2`, `score3`, ...) would be miserable to write and impossible to loop over.

An **array** is a fixed number of values of the same type, stored back to back in memory under one name:

```
int scores[5]; // 5 ints, not set yet
int prices[5] = {90, 85, 77, 60, 100}; // 5 ints, set immediately
```

`scores[5]` reserves 5 slots. `prices[5] = { ... }` reserves 5 slots *and* fills them in, left to right, in the order you list them.

Indexing starts at 0

Each slot in an array has a position number called an **index**, and in C indexing always starts at **0**, not 1. For an array of size 5, the valid indexes are 0, 1, 2, 3, 4 — the last valid index is always `size - 1`, never `size`.

```
printf("%d\n", prices[0]); // 90  <- the FIRST element
printf("%d\n", prices[4]); // 100 <- the LAST element
```

This trips up almost every beginner at least once: reaching for `prices[5]` expecting "the 5th price" and getting the price that doesn't exist. `prices[5]` would be the **6th** slot of a 5-slot array — one past the end. More on why that's dangerous below.

Iterating with a for loop

A `for` loop is the natural match for "do something to every element":

```
for (int i = 0; i < 5; i++) {
    printf("prices[%d] = %d\n", i, prices[i]);
}
```

`i` takes the values 0, 1, 2, 3, 4 — exactly the array's valid indexes — and stops before `i` ever reaches 5. This pattern (`i = 0, i < size, i++`) is the standard way to visit every element of an array, and you'll use it constantly: summing values, finding a maximum, searching for something, printing everything out.

Array bounds: C will not save you

Here's the single most important safety lesson in this module: **C does not check array bounds for you**. Many languages throw a clear, helpful error the instant you try to read or write past the end of an array. C does not. If you write `scores[5]` on a 5-element array, the compiler happily generates code for it — code that reads or writes whatever memory happens to sit right after your array. This is called **undefined behavior**, and the word "undefined" is doing a lot of work: it might print a weird number, it might silently corrupt some *other* variable elsewhere in your program, it might work fine today and crash tomorrow, it might crash immediately. There is no guarantee, and no helpful error message — the bug can show up far away from its actual cause, which makes it one of the hardest kinds of bug to track down.

The discipline that prevents this: **always know your array's real size, and never trust an index blindly**.

```
int size = 5;
int index = get_some_index(); // from input, a calculation, wherever

if (index >= 0 && index < size) {
    printf("%d\n", scores[index]);
} else {
    printf("Index %d is out of bounds.\n", index);
}
```

The classic way this bites beginners is an off-by-one loop condition: writing `i <= size` instead of `i < size` walks one slot too far on the very last pass. Whenever a loop touches an array, double check the condition against the array's actual size.

C-strings: arrays of characters

You already know `char grade = 'A';` — a single character, single quotes. A **string** in C is different: it's an *array* of `char`, written with double quotes, and printed with `%s` instead of `%c`:

```
char grade = 'A';           // ONE character
char name[20] = "Ama";      // a STRING — an array that can hold up to 19
                             // real characters (see why below)
printf("Grade: %c\n", grade);
printf("Name: %s\n", name);
```

Here's the part that catches everyone at first: C secretly stores one extra, invisible character right after "Ama" — `'\0'`, the **null terminator**. It marks "the real string data ends here." So `name[20]` actually looks like this in memory:

```
name[0]='A' name[1]='m' name[2]='a' name[3]='\0' name[4..19]=unused
printf("%s", ...), strlen, and every other string function scan forward until they hit that
'\0' and stop there — they have no other way of knowing where your string ends. This is also why a
char name[20] can only ever hold a string of 19 real characters, not 20: one slot is always reserved
for the terminator. Forget to leave room for it (or overwrite it), and every string function that touches
that array walks straight past your data into undefined behavior.
```

Reading a line of text safely

Module 3 used `scanf("%d", ...)` and `scanf("%f", ...)` for numbers. Reading text with `scanf("%s", ...)` has two serious problems: it stops at the first space (so "Kofi Mensah" becomes just "Kofi"), and it has no idea how big your array is — a long enough input overflows the buffer, which is exactly the out-of-bounds danger from above.

`fgets` fixes both:

```
char name[20];
printf("What is your name? ");
fgets(name, sizeof(name), stdin);
```

`sizeof(name)` tells `fgets` exactly how many bytes it's allowed to write, so it can never overflow the array no matter what the user types. The trade-off: `fgets` keeps the newline character from the Enter key at the end of the string. It's usually not wanted, so it's common to strip it off:

```
name[strcspn(name, "\n")] = '\0';
strcspn(name, "\n") finds the index of the first '\n' in name; writing '\0' there shortens the
string right at that point.
```

Useful `<string.h>` functions

Function	What it does
<code>strlen(s)</code>	Number of real characters in <code>s</code> (not counting <code>'\0'</code>)
<code>strcpy(dest, src)</code>	Copies <code>src</code> into <code>dest</code> (<code>dest</code> must be big enough)
<code>strcmp(a, b)</code>	Returns 0 if <code>a</code> and <code>b</code> are exactly equal

Function	What it does
<code>strcat(dest, src)</code>	Appends <code>src</code> onto the end of <code>dest</code>

```
char city[20] = "Kumasi";
printf("%zu\n", strlen(city));           // 6

char copy[20];
strcpy(copy, city);                     // copy now holds "Kumasi"

if (strcmp(copy, city) == 0) {           // true - same contents
    printf("They match!\n");
}

char greeting[40] = "Welcome to ";
strcat(greeting, city);                 // greeting is now "Welcome to Kumasi"
```

Notice `strcmp`, not `==`. Comparing two strings with `==` compares where they live in memory, not what characters they contain — it will almost never do what you want. Always reach for `strcmp` instead.

A preview: 2D arrays

Some problems need a grid instead of a single row — a tic-tac-toe board, a seating chart, a spreadsheet of numbers. C lets you declare a **2D array** for exactly that:

```
int grid[3][3]; // 3 rows, each holding 3 ints
```

You won't need these yet — plain 1D arrays are enough for this module — but it's worth knowing the idea exists for when a problem needs more than one dimension.

One more preview, just so the name doesn't surprise you later: an array name like `scores` is closely related to something called a **pointer** — you'll see exactly what that means and how it works in Module 7.

Common beginner mistakes

- Using index `size` instead of `size - 1` for the last element (off-by-one), or looping with `i <= size` instead of `i < size`.
- Reading or writing an index outside the array's bounds — C will not warn you; the bug can show up somewhere completely unrelated later on.
- Declaring a `char` array too small for the string plus its `'\0'` (e.g. `char name[3] = "Ama";` — that needs at least 4 slots, not 3).
- Using `scanf("%s", ...)` for user input instead of `fgets` — it splits on whitespace and has no protection against buffer overflow.
- Comparing strings with `==` instead of `strcmp`.
- Forgetting that `fgets` keeps the trailing newline, and being confused when a "matching" string comparison mysteriously fails.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one.
2. Complete **exercises/exercisel.c**.
3. Build the **module project**.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Module 5: arrays and strings"  
git push
```

Next: **Module 6 — Functions.**

Module 6 — Functions

Hook: the same receipt, two very different mains

Compile and run both of these — don't read ahead first.

Version A:

```
#include <stdio.h>

int main(void) {
    double price1 = 4.50;
    int qty1 = 3;
    double subtotal1 = price1 * qty1;
    double tax1 = subtotal1 * 0.15;
    double total1 = subtotal1 + tax1;
    printf("Coffee: %d x $%.2f = $%.2f (tax $%.2f) => total $%.2f\n",
           qty1, price1, subtotal1, tax1, total1);

    double price2 = 2.25;
    int qty2 = 5;
    double subtotal2 = price2 * qty2;
    double tax2 = subtotal2 * 0.15;
    double total2 = subtotal2 + tax2;
    printf("Muffin: %d x $%.2f = $%.2f (tax $%.2f) => total $%.2f\n",
           qty2, price2, subtotal2, tax2, total2);

    return 0;
}
```

Version B:

```
#include <stdio.h>

void printReceiptLine(char itemName[], double unitPrice, int quantity) {
    double subtotal = unitPrice * quantity;
    double tax = subtotal * 0.15;
    double total = subtotal + tax;
    printf("%s: %d x $%.2f = $%.2f (tax $%.2f) => total $%.2f\n",
           itemName, quantity, unitPrice, subtotal, tax, total);
}

int main(void) {
    printReceiptLine("Coffee", 4.50, 3);
    printReceiptLine("Muffin", 2.25, 5);
    return 0;
}

gcc versionA.c -o versionA && ./versionA
gcc versionB.c -o versionB && ./versionB
```

Same numbers in, same receipt lines out — byte for byte. Version B is less than half the length, and if the tax rate ever changes, you fix it in exactly **one** place instead of hunting down every copy-pasted block. That's the entire pitch for this module: a **function** is a named, reusable chunk of code you write once and use as many times as you like. By the end of this module you'll refactor a much bigger program the same way. You can see this exact comparison, with a bit more detail, in

examples/01_why_functions_before_and_after.c.

Why functions?

Think back to your Module 4 Number Guessing Game project: the computer picks a random number, and `main()` loops — reading a guess, checking whether it's in range, comparing it to the secret number, and printing "too high," "too low," or "correct." For a first version, all of that living inside one `main()` is fine.

But picture growing that game a little: add a limit on the number of guesses, add a "play again?" option, add a difficulty setting that changes the range. Every new feature means `main()` gets a few lines longer and a bit more tangled — the input-reading code, the validation code, the comparison code, and the printing code all interleaved. Six months from now (or six minutes from now, debugging it), it's hard to answer a simple question like "where exactly do we decide the player won?" because that logic is mixed in with everything else.

Functions fix this by giving each *job* a name and a boundary. "Get a valid guess from the player" becomes a function. "Check the guess" becomes a function. `main()` shrinks down to reading like a to-do list: get a guess, check it, print the result, repeat. You'll do exactly this refactor to your own guessing game in this module's project.

Breaking a program into functions gives you three things at once:

- **Reusability** — write the logic once, call it from anywhere, as many times as you like (that's the receipt hook, above).
- **Readability** — a well-named function tells you what a block of code does without you having to read its body. `checkGuess(guess, secret)` is self-explanatory; twelve lines of `if/else` inline in `main()` aren't, at a glance.
- **Testability** — you can try a function on its own, with known inputs, and check it gives the right output, without running your whole program end to end.

Declaring and calling a function

A function has three parts: a **header** (what it's called, what it takes in, what it gives back), a **body** (the code that runs), and, separately, every place you **call** it.

```
void printStarRow(int count) {
    for (int i = 0; i < count; i++) {
        printf("*");
    }
    printf("\n");
}

int main(void) {
    printStarRow(3);
    printStarRow(8);
    return 0;
}
```

Reading the header left to right: `void` is the **return type** (this function doesn't send any value back — more on that soon), `printStarRow` is the **name**, and `(int count)` is the **parameter list** — the inputs this function expects. Every time you write `printStarRow(3)` ;, you're **calling** the function, and 3 flows into `count` for that one run.

See `examples/02_declaring_and_calling.c`.

Prototypes: telling the compiler what's coming

C reads your file from top to bottom, once. If `main()` calls a function that hasn't been defined yet — because its full definition is further down the file — the compiler has no idea it exists and refuses to compile.

A **prototype** (also called a **declaration**) solves this. It's the function's header line, on its own, ending in a semicolon, with no body:

```
void describeTemperature(int celsius); // prototype

int main(void) {
    describeTemperature(35); // compiler already knows this exists
    return 0;
}

void describeTemperature(int celsius) { // full definition, further down
    if (celsius >= 30) {
        printf("%d C: that's hot.\n", celsius);
    } else {
        printf("%d C: that's not so hot.\n", celsius);
    }
}
```

The prototype is a promise: "a function with this exact name, these parameter types, and this return type exists somewhere in this file." The compiler holds you to it — if the real definition doesn't match, or never shows up, you'll get an error at link time. This is exactly why you'll often see a block of prototypes near the top of a C file, above `main()`: it lets you write `main()` first, reading top-to-bottom like a summary of the program, with the messy implementation details below it.

See `examples/03_prototypes.c`.

Parameters, arguments, and return values

These two words get mixed up constantly, so pin them down now:

- A **parameter** is the named placeholder in the function's own header — it's part of how the function is *written*.
- An **argument** is the actual value you hand over at the *call site*.

```
int add(int a, int b) { // a and b are PARAMETERS
    return a + b;
}

int total = add(3, 4); // 3 and 4 are ARGUMENTS
```

A function can send a result back to its caller with `return`. The type before the function's name promises what type that will be:

```
int isEven(int number) {
    if (number % 2 == 0) {
        return 1;
    } else {
        return 0;
    }
}
```

`return` immediately exits the function with that value — any code after a `return` in the same path never runs. You can use the returned value directly (`printf("%d\n", isEven(7));`) or store it in a variable first.

See `examples/04_parameters_and_return_values.c`.

void functions

Not every function needs to hand a value back. A function whose job is just to *do* something — print a banner, update the screen, play a sound — declares `void` as its return type, meaning "nothing comes back." You already met one above: `printStarRow`. A `void` function can still use a bare `return`; (no value) to exit early, but it's optional.

Scope: where a variable lives

A variable declared inside a function only exists **inside that function**, for as long as that function is running. This is called its **scope**. Once the function returns, its local variables are gone — nothing outside can see them, read them, or accidentally clash with a variable of the same name in a different function.

```
void printWelcomeBanner(char userName[]) {
    int line = 1;           // exists only inside this function
    printf("%d) Welcome, %s!\n", line, userName);
}

int main(void) {
    printWelcomeBanner("Ama");
    // printf("%d\n", line); // ERROR: `line` doesn't exist out here
    return 0;
}
```

A **global variable** — declared outside every function — is the opposite: every function can read and change it. It compiles, and sometimes you'll see it in other people's code, but avoid it as a beginner. When any function anywhere can silently change a shared variable, tracking down *which* function caused a wrong value becomes a real headache as programs grow. Passing values in as parameters and getting results back with `return` keeps each function's effects visible and contained.

See `examples/05_void_functions_and_scope.c`.

Passing an array to a function

You can pass an array to a function just like any other value — you just also pass its size as a separate argument, since C doesn't track an array's length for you once it's inside the function:

```
int findMax(int arr[], int size) {
    int max = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}
```

Here's the part that's genuinely different from every other type you've passed to a function so far: **when you pass an array, the function can modify the original array back in `main()`** — not just its own private copy. An `int` or `double` argument gets copied in, and changes inside the function stay inside the function. Arrays don't work that way. You'll find out exactly why in Module 7 — for now, just know arrays are the exception, and treat any function that takes an array as a function that might change it.

See `examples/06_passing_arrays_to_functions.c`.

Refactoring: turning a mess into functions

Refactoring means changing how code is organized without changing what it does. It's the single most practical skill in this module. The process is always the same:

1. Find a self-contained piece of logic — something you could describe in one short sentence ("this part checks if the guess is too high or too low").
2. Give that sentence a function name (`checkGuess`).
3. Move the code into a new function with that name, turning the values it depended on into parameters, and whatever it produced into a `return` value (or a `printf`, if its whole job is printing).
4. Replace the original code with a call to your new function.
5. **Recompile and test immediately.** Refactoring should never change behavior — if the output changes, you introduced a bug, not an improvement.

That's exactly what you did conceptually in the hook at the top of this module (turning duplicated receipt math into `printReceiptLine`), and it's exactly what this module's project asks you to do to your whole Number Guessing Game.

Common beginner mistakes

- Forgetting a function's **prototype** when it's defined below `main()` — the compiler will complain about an undeclared or implicitly declared function, often pointing at the *call*, not the missing prototype.
- Mixing up **parameters** and **arguments** in conversation — harmless day to day, but know the difference when reading error messages.
- Writing `void` on a function and then trying to use its result: `int x = printStarRow(3);` won't compile — a `void` function has no value to give back.
- Forgetting `return` in a non-`void` function, or returning the wrong type. The compiler will often warn you, but always double check.
- Trying to use a local variable outside the function it was declared in — remember, its scope ends at the function's closing `}`.
- Forgetting to pass an array's size as its own parameter, or assuming a function won't change the array you passed it, when it does.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one.
2. Complete **exercises/exercisel.c**.
3. Build the **module project**.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Module 6: functions"  
git push
```

Next: **Module 7 — Pointers**.

Module 7 — Pointers

Hook: the magic swap that isn't

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>

void swapBroken(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
    printf("    inside swapBroken: a = %d, b = %d\n", a, b);
}

int main(void) {
    int x = 3;
    int y = 7;

    printf("Before: x = %d, y = %d\n", x, y);
    swapBroken(x, y);
    printf("After:  x = %d, y = %d\n", x, y);

    return 0;
}
gcc swap.c -o swap
./swap
```

Look closely at the output. Inside `swapBroken`, `a` and `b` swap just fine — but back in `main`, `x` and `y` are completely unchanged. The function did exactly what you told it to. So why didn't it work?

The answer is coming in this module, and once you have it, you'll fix this with a real, working swap using nothing but the tools below. This is also the exact question that's been waiting since Module 6: why could a function change the contents of an array you passed it, but never change a plain `int`, `float`, or `char` the same way? Same answer.

What is a memory address, really?

Every variable you create has to physically live somewhere while your program runs — in **RAM (random-access memory)**. You can think of RAM as an enormous row of numbered mailboxes; when you write `int age = 25;`, the compiler picks a free mailbox, labels it `age` (just for your convenience while reading the code), and stores 25 inside it. The *number* of that mailbox is called its **address**.

You've already been using addresses without necessarily thinking of them that way — every time you wrote `scanf("%d", &age);` back in Module 3, that `&` meant "don't give `scanf` the value of `age`, give it the *address* of `age`, so `scanf` can reach in and store what the user typed directly into that mailbox." That's the whole mystery behind `&` finally explained.

You can print any variable's address yourself, using `&` and the `%p` format specifier:

```
int age = 25;
printf("age lives at address: %p\n", (void *)&age);
```

The address itself will look like a strange string of hex digits (e.g. `0x7ffee3a1c9ac`). You will never need to memorize or predict what it says — what matters is that it exists, and that a **pointer** is simply a variable built to store one.

Declaring a pointer

A pointer is a variable, just like any other — except instead of holding a number or a character, it holds an *address*. You declare one with a `*` between the type and the name:

```
int *p;
```

Read this as "p is a pointer to `int`" — meaning: whatever address `p` holds, it promises that address contains an `int`. This matters, because the computer needs to know how many bytes to read starting from that address; a pointer to `int` and a pointer to `char` read different amounts of memory when you use them.

The dual meaning of `*` — the single biggest beginner trap

Here is the part that trips up almost everyone at first: the `*` symbol means **two completely different things** depending on where it shows up, and C gives you no visual hint about which one you're looking at.

```
int *p = &age;    // (1) declaration: * means "p is a pointer"
*p = 100;         // (2) dereference: * means "the value at the address p
holds"
```

- In line (1), `*` is part of *declaring* `p`'s type. It only appears once, at the moment you introduce the variable.
- In line (2), `*` is the **dereference operator**. Applied to a pointer that already exists, `*p` means "go to the address stored in `p`, and give me (or let me set) the value that lives there."

The rule of thumb: if you're introducing a brand-new variable name right after a type, `*` is part of the declaration. Anywhere else, `*` is asking to follow the pointer to what it points to.

Assigning and dereferencing a pointer

Putting declaration and dereference together:

```
int age = 25;
int *p = &age;    // p now holds the ADDRESS of age -- "p points to age"

printf("%d\n", *p); // dereference: "the value at the address p holds" -> 25
```

`p` and `age` are two different variables. `p` doesn't contain 25 — it contains *the address where 25 lives*. `*p` is how you ask C to follow that address and hand you what's actually stored there.

Modifying a value through a pointer

Because `*p` means "the value at the address `p` holds," you can also *assign* through it:

```
*p = 100; // go to the address p holds, and store 100 there
printf("%d\n", age); // prints 100 -- age itself changed!
```

`p` never changed — it still points to the same address. What changed is the value **sitting at** that address, which happens to be `age`. This is the mechanism that makes pointers useful: they let one part of your program reach out and change a variable that belongs to another part.

Pass-by-value vs. pass-by-pointer: solving the hook

Back to `swapBroken`. When you call `swapBroken(x, y)`, C copies the *values* of `x` and `y` into brand-new local variables `a` and `b`. Swapping `a` and `b` swaps two copies that live nowhere near `x` and `y` — and disappear the moment the function returns. This is called **pass-by-value**, and it's how every plain parameter (`int`, `float`, `char`, ...) works in C, always.

To actually reach `x` and `y`, you must give the function their *addresses* instead — pass-by-pointer:

```
void swapWorks(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// called as:
swapWorks(&x, &y);
```

Now `a` holds the address of `x`, and `b` holds the address of `y`. `*a` and `*b` reach all the way back into `main`'s variables and swap the real thing. Compile and run this version — `x` and `y` really do swap this time.

C has no separate "pass by reference" keyword the way some other languages do. **Pointers are how C does it** — explicitly, by you writing `&` at the call site and `*` inside the function.

Arrays and pointers: Module 6's mystery, solved

In Module 6, you learned that a function *can* modify the caller's array directly — unlike a plain `int`. Now you know why: when you pass an array to a function, it **decays into a pointer to its first element**. You were never passing a copy of the whole array — you were passing an address, exactly like `&x` above, whether you wrote `&` or not.

That means these two function signatures are interchangeable:

```
int findMax(int arr[], int size) { ... } // Module 6's version
int findMax(int *arr, int size) { ... } // exactly the same function
```

Both receive the address of the array's first element. `arr[i]` still works either way — it's just friendlier notation for "the value `i` slots past the address `arr` holds."

NULL pointers

A pointer that isn't pointing at a real variable yet should be set to `NULL` — a special value meaning "this points to nothing, on purpose."

```
int *p = NULL;

if (p != NULL) {
    printf("%d\n", *p);
} else {
    printf("p isn't pointing anywhere yet.\n");
}
```

Always check for `NULL` before dereferencing a pointer you're not certain has been pointed at something. Dereferencing `NULL` — or any uninitialized "garbage" pointer that was never assigned an address at all — is undefined behavior. Most often it crashes your program immediately (a "segmentation fault"); sometimes, worse, it silently reads or corrupts memory that isn't yours and doesn't crash at all.

Common beginner mistakes

- Forgetting `&` when a function needs an address (e.g. `swapWorks(x, y)` instead of `swapWorks(&x, &y)`) — this is usually a compiler warning about mismatched types, not silence, so read those warnings.
- Confusing declaration-`*` with dereference-`*` — remember: only at the moment a pointer variable is introduced does `*` mean "this is a pointer type."
- Dereferencing a pointer that was never assigned an address (garbage) or was set to `NULL` — always check `if (p != NULL)` first when you're not certain.
- Assuming a pointer and the variable it points to are "the same variable" — they're two separate variables; one just happens to store the other's address.
- A sharp edge for later: `*p++` does NOT mean what it looks like at a glance — `++` binds tighter than `*`, so precedence rules matter a lot once you start combining pointer arithmetic with increment/decrement. You don't need to master this now; just know it's coming, and use parentheses like `(*p)++` when in doubt.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one.
2. Complete **exercises/exercise1.c**.
3. Build the **module project**.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 7: pointers"
git push
```

Next: **Module 8 — Structs & File I/O.**

Module 8 — Structs & File I/O

Hook: a program that remembers

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>

typedef struct {
    int highScore;
} GameData;

int main(void) {
    GameData data;
    FILE *file = fopen("save.txt", "r");

    if (file == NULL) {
        printf("No save file found -- starting fresh.\n");
        data.highScore = 0;
    } else {
        fscanf(file, "%d", &data.highScore);
        fclose(file);
        printf("Welcome back! Your previous high score was %d.\n",
data.highScore);
    }

    int newScore;
    printf("Enter your score this round: ");
    scanf("%d", &newScore);

    if (newScore > data.highScore) {
        data.highScore = newScore;
        printf("New high score!\n");
    }

    file = fopen("save.txt", "w");
    if (file == NULL) {
        printf("Error: could not save your score.\n");
        return 1;
    }
    fprintf(file, "%d", data.highScore);
    fclose(file);

    printf("High score (%d) saved. Run this program again and see for
yourself.\n", data.highScore);
    return 0;
}
gcc hook.c -o hook
./hook
```

Enter a score, let it tell you it saved a new high score, then **run `./hook` again**. This time it greets you by name of achievement — "Welcome back! Your previous high score was..." — using data from a run of the program that already ended. Every variable from that first run is long gone from memory; only the *file* survived. By the end of this module you'll understand every line of this, and you'll build a bigger version of the same trick in the module project.

Grouping data that belongs together

Modules 1–7 gave you variables, arrays, and functions, but every array you've built so far holds only ONE kind of thing: an array of `int`, an array of `char`. Real data doesn't come in single columns — a student has a name AND an age AND a GPA, all at once, all describing the same person.

Without structs, you'd track that with **parallel arrays**:

```
char names[3][30] = {"Ama", "Kwame", "Efua"};
int ages[3]       = {20, 22, 21};
float gpas[3]     = {3.8f, 3.2f, 3.9f};
```

This works, but it's fragile: `names[i]`, `ages[i]`, and `gpas[i]` only describe "the same student" because you remember to keep all three arrays in sync by hand. Sort one and forget the other two, and Ama's name silently ends up next to Kwame's age. The compiler has no idea these three arrays are related, so it can't warn you when they drift apart.

Defining a struct

A **struct** groups related variables — called **members** — into a single new type:

```
struct Student {
    char name[30];
    int age;
    float gpa;
};
```

This doesn't create a variable yet — it defines a new *type* called `struct Student`, the same way `int` or `char` is a type. To actually get a variable, declare one like anything else:

```
struct Student s1;
```

Accessing members with .

Use the **dot operator** (`.`) to read or set a member of a struct variable:

```
strcpy(s1.name, "Ama"); // char arrays still need strcpy (Module 5)
s1.age = 20;
s1.gpa = 3.8f;

printf("%s is %d\n", s1.name, s1.age);
```

You can also fill in every member at once, in declaration order:

```
struct Student s2 = {"Kwame", 22, 3.2f};
```

typedef: a shorter name for the same type

Typing `struct Student` every time you declare a variable gets old fast. `typedef` lets you attach a shorter alias to the type:

```
typedef struct {
    char name[30];
    int age;
    float gpa;
} Student;
```

```
Student s1; // instead of: struct Student s1;
```

This is **purely a naming convenience** — `Student` and the `struct` it names are exactly the same type underneath, using exactly the same amount of memory, with exactly the same members. `typedef` just saves you keystrokes and reads a little cleaner. From here on, this module uses the `typedef` style.

Arrays of structs

The whole point of fixing the parallel-arrays problem: one array, where each slot is a *complete* student.

```
Student class[3] = {
    {"Ama", 20, 3.8f},
    {"Kwame", 22, 3.2f},
    {"Efua", 21, 3.9f}
};

for (int i = 0; i < 3; i++) {
    printf("%s is %d, GPA %.1f\n", class[i].name, class[i].age, class[i].gpa);
}
```

Looping looks exactly like the array loops from Module 5 — you're just reaching one member deeper with `.` on each element.

Structs and pointers: the arrow operator

Module 7 taught you `&` (address-of) and `*` (dereference). Both still work exactly the same way on structs:

```
Student s1 = {"Ama", 20, 3.8f};
Student *p = &s1;

printf("%d\n", (*p).age); // dereference p, THEN access .age
```

The parentheses around `*p` are required — `.` binds tighter than `*`, so `*p.age` would (incorrectly) try to do `p.age` first. Because `(*p).member` is so common, C gives you a dedicated operator for it: the **arrow operator**, `->`.

```
printf("%d\n", p->age); // exactly the same thing as (*p).age
```

`p->age` and `(*p).age` mean *identically* the same thing — always prefer `->`, it's what every C programmer expects to read. And because `p` *points at* `s1` rather than holding a separate copy, writing through the pointer really does change the original:

```
p->age = 21;
printf("%d\n", s1.age); // prints 21 -- same variable
```

Passing structs to functions by pointer

Module 7's biggest lesson was: passing a variable to a function normally passes a *copy*, so changes inside the function don't escape it — unless you pass a pointer instead. That rule applies to structs exactly as it applies to `int`:

```

void birthdayByValue(Student s) {
    s.age = s.age + 1;    // only changes the LOCAL copy
}

void birthdayByPointer(Student *s) {
    s->age = s->age + 1;  // changes the CALLER's struct
}

int main(void) {
    Student s1 = {"Ama", 20, 3.8f};
    birthdayByValue(s1);
    printf("%d\n", s1.age);    // still 20

    birthdayByPointer(&s1);
    printf("%d\n", s1.age);    // now 21
}

```

Rule of thumb: pass a struct **by pointer** whenever a function needs to *modify* the caller's copy — `&s1` at the call site, `Student *s` as the parameter, `s->member` inside the function.

Basic file I/O

Every program you've written so far forgets everything the moment it exits — all its variables live in memory, and memory is wiped clean when the process ends. To make data survive, you have to write it to a **file** on disk.

Opening a file: *fopen*

```
FILE *file = fopen("students.txt", "w");
```

`fopen` takes a filename and a **mode** string describing what you intend to do:

Mode	Meaning
"w"	Write. Creates the file if missing; erases existing contents if it already exists.
"r"	Read. Fails if the file doesn't exist.
"a"	Append. Like "w", but adds to the <i>end</i> of an existing file instead of erasing it.

Always check for *NULL*

`fopen` can fail — the file doesn't exist (in "r" mode), a bad path, no permission to write there. When it fails, it returns `NULL` instead of a usable `FILE*`. **Always** check before using the result:

```

if (file == NULL) {
    printf("Error: could not open the file.\n");
    return 1;
}

```

Skipping this check is one of the most common real-world C bugs: the program crashes the instant it tries to read or write through a `NULL` file pointer, often with a confusing error far from the real cause.

Writing: *fprintf*

Works exactly like `printf`, except the first argument says *where* the text goes:

```
fprintf(file, "%s %d %.2f\n", s.name, s.age, s.gpa);
```

Reading back: *fscanf*

Works exactly like `scanf` (Module 3), except it reads from a file instead of the keyboard. It returns how many values it successfully matched, which is exactly what you need to loop until the file runs out:

```
Student s;
while (fscanf(file, "%29s %d %f", s.name, &s.age, &s.gpa) == 3) {
    // got one full student -- store it, e.g. class[count++] = s;
}
```

Closing: *fclose* — and why it matters

```
fclose(file);
```

Text you write with `fprintf` isn't necessarily on disk the instant you call it — it can sit in a memory buffer for efficiency, and only actually get written when the buffer fills up, the program ends normally, or you close the file. `fclose` forces that write to happen *now*, and releases the file so nothing else is blocked from using it. A program that writes data but crashes (or is killed) before closing its files can lose data it thinks it already saved. Get in the habit: every successful `fopen` gets a matching `fclose`.

Putting it together

The hook at the top of this module is the whole pattern in miniature: write struct data to a file with `fprintf` in one run, read it back with `fscanf` into a fresh array in a completely separate run, and the data behaves as if the program never forgot it. **examples/06_writing_a_file.c** and **examples/07_reading_a_file.c** show this with a full array of structs instead of a single number — run 06 first, then 07, and watch the array come back exactly as you saved it.

This is also exactly what **Module 9's capstone project** requires: a struct type stored in an array, saved and reloaded so the program's data survives being closed and reopened. Everything in this module exists to make that requirement feel routine by the time you get there.

Common beginner mistakes

- Forgetting to check `fopen`'s return value for `NULL` before using it — this crashes the program the moment the file doesn't exist or can't be opened.
- Forgetting `fclose` — your data may never actually make it to disk, or later attempts to open the same file may behave unexpectedly.
- Opening a file in "w" mode when you meant "a" (or vice versa) and accidentally erasing everything that was there.
- Writing `*p.age` instead of `(*p).age` or `p->age` — `.` binds tighter than `*`, so this doesn't do what it looks like it does.
- Mismatching the `fscanf` format string with what `fprintf` actually wrote (wrong order, wrong number of fields, or a name containing a space when `%s` stops at the first space) — read the file back with a text editor if a read isn't behaving as expected.
- Forgetting the size limit in `%29s`-style reads into a fixed-size `char` array, allowing a read to overflow the array (Module 5's string-safety lesson still applies here).

Try it yourself

1. Work through every file in **examples/**, compiling and running each one. Run `06_writing_a_file.c`, then `07_reading_a_file.c` right after it, in the same folder — you're proving to yourself that the second run really does read what the first run wrote.
2. Complete **exercises/exercisel.c**.
3. Build the **module project** — run it, save, quit, and run it again to prove your data survived.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Module 8: structs and file I/O"  
git push
```

Next: **Module 9 — Capstone Project.**

Module 9 — Capstone Project

This is where every module comes together: variables, control structures, arrays, strings, functions, pointers, structs, and file I/O, combined into one real program you'll be proud to link from your GitHub profile.

1. How teams are formed

Your instructor announces the class size before this module starts, which sets your group size:

Class size	Group size	Ratio
Up to 20 students	Pairs	2:1
More than 20 students	Groups of 4	4:1
Approved solo ("daring") students, any class size	Individual	1:1

Going solo ("daring" track): any student may request to build the capstone alone instead of in a group. This isn't a smaller workload — it's the same core requirements (§3) **plus every stretch goal** listed for your chosen project (§4), since you don't have teammates splitting the work. Request this from your instructor *before* Sprint 0 (§5) so GitHub Classroom can set up your repo correctly. Going solo is a good fit if you're already comfortable with everything through Module 8 and want the extra stretch — it is not a reward for wanting to avoid teammates.

2. Forming your team in GitHub Classroom

1. Your instructor posts **one Group Assignment invite link** for the capstone.
2. Open it, sign in with GitHub, and either **create a new team** (name it after your project idea, e.g. `library-system-team1`) or **join an existing team** a teammate already created. Team size is enforced by the size from the table above — Classroom won't let you overfill it.
3. Solo/"daring" students accept the same link but create a team of one.
4. GitHub Classroom provisions **one shared repository** for your team, copied from the capstone starter template. Every teammate clones the *same* repo:

```
git clone https://github.com/YOUR-CLASSROOM-ORG/capstone-yourteamname.git
cd capstone-yourteamname
```

1. Everyone commits under their **own** Git identity (`git config user.name/user.email` from Module 0) — this is how your instructor sees who contributed what. A repo with every commit from one teammate and none from the other is a problem worth fixing early, not at the deadline.

3. Requirements for every project (any topic, any team size)

Regardless of which idea you pick from §4, your submission must include:

- **At least one struct type**, used in an array or similar collection (Module 8) — this is your program's core data.
- **File I/O** (Module 8): your program's data must survive being closed and reopened. If you quit and relaunch, your data should still be there.
- **At least 4 functions you wrote yourself** (Module 6), each doing one clear job — not one giant `main`.

- **Pointers used with intention** (Module 7) somewhere real — e.g. passing a struct by pointer to modify it, not just because it's required. Be ready to explain why you used a pointer where you did.
- **Arrays and/or strings** (Module 5) for any collection of data or text processing.
- **Control structures and a menu-driven loop** (Module 4) — the program should run continuously, present options, and only exit when the user chooses to.
- **A project README .md** in your team's repo explaining: what the program does, how to compile and run it, and a one-paragraph description of who worked on what.
- **No compiler warnings** with `gcc -Wall yourfile.c -o yourprogram`. Warnings are gcc telling you about likely bugs — clear them all.

4. Choose a project

Pick one. All of them are scoped to fit the requirements above with room for your own ideas on top.

1. **Student Grading & Report Card System** — store students (struct: name, ID, array of scores), compute averages/grades, save/load the class roster to a file. *Stretch*: rank students, export a formatted report per student.
2. **Library / Inventory Management System** — track items (struct: title, ID, quantity, checked-out status), check items in/out, persist the catalog to a file. *Stretch*: search by keyword, track due dates.
3. **Contact Book / Phone Directory** — add, search, edit, and delete contacts (struct: name, phone, email), saved to a file. *Stretch*: sort alphabetically, group by category.
4. **Quiz Game Engine** — load a question bank from a file, ask questions, score the player, save high scores. *Stretch*: multiple difficulty levels, a timer.
5. **Simple Bank Account Simulator** — deposit, withdraw, check balance, with a persisted transaction log (struct per transaction). *Stretch*: multiple accounts, interest calculation, transaction history search.
6. **Text-Based Adventure Game** — rooms and choices driving the player through a small story, with save/load progress. *Stretch*: an inventory system using structs, multiple endings.
7. **Personal Expense Tracker** — log expenses (struct: category, amount, date) to a file, show totals and simple statistics. *Stretch*: monthly summaries, a simple budget-vs-actual comparison.

Have your own idea? Propose it to your instructor — as long as it can satisfy every requirement in §3, it's fair game.

5. Suggested timeline

Sprint	Focus	Git evidence expected
Sprint 0	Team formed, project chosen, <code>structs</code> designed on paper first	An initial commit with your struct definitions and a plan in the README
Sprint 1	Core data operations working (add/search/edit) in memory, no file I/O yet	Several small commits, each compiling cleanly
Sprint 2	File I/O: save and load your data correctly	A commit where you can close and reopen the program without losing data

Sprint	Focus	Git evidence expected
Sprint 3	Menu loop, polish, edge cases (empty list, bad input)	Commits addressing specific bugs/edge cases you found
Final	Clean <code>-Wall</code> build, finished README, last push before deadline	Your final commit <i>is</i> your submission

Commit at the end of every session you work, even if it's not finished — a commit history of one giant commit the night before the deadline is a red flag, not a shortcut.

6. How this gets assessed

- **Correctness** — does it do what it claims, including edge cases (empty input, invalid menu choice, saving/loading correctly)?
- **Use of the module concepts** — structs, pointers, functions, arrays, and file I/O used appropriately, not bolted on to satisfy a checklist.
- **Code organization** — logically split into functions, consistent style, comments where the *why* isn't obvious.
- **Git history** — regular, meaningful commits from every team member under their own name.
- **Documentation** — a README a stranger could use to build, run, and understand your project without asking you anything.

Try it yourself

1. Confirm your team (or solo status) in GitHub Classroom.
2. Clone your team's repo.
3. Pick a project from §4 (or propose your own).
4. Work through the sprints in §5, committing and pushing regularly.
5. Submit by pushing your final commit before the deadline — there is no separate submission step.

You've built something real. Put the repo link on your CV.

Part 2 — Intermediate C (Modules 10–19)

The actual dividing line between “I can write C” and “I’ve written some C once”: recursion, dynamic memory, linked lists and the structures built on them, multi-file program organization, debugging tools, and a second, harder capstone.

Module 10 — Recursion

Part 2 begins here. Modules 10–19 assume you've finished Part 1 (Modules 0–9) — everything from `printf` through pointers, structs, and file I/O is fair game and won't be re-explained.

Hook: a countdown with no loop anywhere in sight

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>

void countdown(int n) {
    if (n == 0) {
        printf("Liftoff!\n");
        return;
    }
    printf("%d...\n", n);
    countdown(n - 1);
}

int main(void) {
    countdown(5);
    return 0;
}

gcc hook.c -o hook
./hook
```

5... 4... 3... 2... 1... Liftoff! — six lines of output, and there is no `for`, no `while`, no `do-while` anywhere in this file. `countdown` is doing something new: **calling itself**. By the end of this module you'll understand exactly how a function can repeat work without a loop, and you'll build a small recursive ASCII-art generator in this module's project.

What is recursion?

A **recursive function** is a function that calls itself, working on a smaller version of the same problem each time, until it reaches a version small enough to answer directly. Every recursive function needs exactly two parts:

- **The base case** — the smallest version of the problem, answered directly, with no further recursive call. In the hook, that's `n == 0`: just print "Liftoff!" and stop.
- **The recursive case** — do a small piece of work, then call the same function again on a problem that's *closer* to the base case. In the hook, that's "print `n`, then countdown from `n - 1`."

Miss the base case, or write a recursive case that never actually gets closer to it, and the function calls itself forever — see **Common beginner mistakes** below.

Tracing the call stack

Every function call — recursive or not — gets its own private copy of its parameters and local variables, stored in a **stack frame**. Normal function calls (Module 6) use exactly one frame at a time. Recursion is different: each recursive call adds a *new* frame on top of the last one, and none of those frames go away until the calls start **returning**, one at a time, from the innermost call outward.

Trace `countdown(3)` by hand:

```

countdown(3) called
  prints "3..."
  calls countdown(2)
    countdown(2) called
      prints "2..."
      calls countdown(1)
        countdown(1) called
          prints "1..."
          calls countdown(0)
            countdown(0) called -- base case! prints "Liftoff!", returns
            countdown(1) resumes after its call, has nothing left to do, returns
            countdown(2) resumes, has nothing left to do, returns
            countdown(3) resumes, has nothing left to do, returns

```

Four separate stack frames existed briefly, all at once, each remembering its own value of n — that's why $n - 1$ in each call doesn't disturb any other call's n . See

examples/01_countdown_no_loop.c — it's the hook with a print statement added on the way back out, so you can watch the unwinding happen.

A classic example: factorial

`factorial(n)` (written $n!$) is $n \times (n-1) \times (n-2) \times \dots \times 1$, and it translates almost word-for-word into a recursive definition: " $n!$ is n times $(n-1)!$ — unless n is 0, in which case the answer is just 1."

```

int factorial(int n) {
    if (n == 0) {           // base case
        return 1;
    }
    return n * factorial(n - 1); // recursive case
}

```

Every recursive call here waits for the call *below* it to return before it can finish its own multiplication — `factorial(4)` can't produce a final answer until `factorial(3)` returns, which can't finish until `factorial(2)` returns, and so on down to the base case. See

examples/02_factorial_recursive.c.

Recursion over an array: sum of elements

Recursion isn't limited to counting down a single number — it works over arrays too, by recursing on "everything except the first element":

```

int sumArray(int arr[], int size) {
    if (size == 0) {           // base case: an empty array sums to 0
        return 0;
    }
    return arr[0] + sumArray(arr + 1, size - 1);
}

```

`arr + 1` is pointer arithmetic from Module 7 — it points one element further into the array, so each recursive call sees one fewer element, moving steadily toward the base case. See

examples/03_sum_of_array_recursive.c.

Recursive vs. iterative: two ways to solve the same problem

Anything you can write recursively, you can also write with a loop — `factorial` is a good side-by-side:

```
int factorialIterative(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++) {
        result = result * i;
    }
    return result;
}
```

Neither version is "more correct." Loops are usually a little more memory-efficient (one stack frame instead of many) and can be easier to follow for simple counting. Recursion tends to shine when a problem is *naturally* defined in terms of a smaller version of itself — as you'll see clearly in Module 12 (Linked Lists), where walking a chain of nodes maps onto recursion almost perfectly. For now, the honest beginner rule: **if a loop solves it cleanly, use the loop — reach for recursion when the problem genuinely describes itself in smaller pieces.** See `examples/04_recursive_vs_iterative.c`.

A cautionary tale: naive Fibonacci

Not every recursive solution is a *good* one. The Fibonacci sequence (0, 1, 1, 2, 3, 5, 8, 13, . . ., each number the sum of the two before it) has an obvious-looking recursive definition:

```
int fib(int n) {
    if (n <= 1) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}
```

This is correct — and, past roughly $n = 35$, painfully slow. Trace it by hand for `fib(5)` and you'll notice `fib(3)` gets computed **twice**, `fib(2)` three times, and it only gets worse as n grows — the same work is repeated over and over instead of reused. This isn't a reason to distrust recursion; it's a reason to notice when a recursive definition is quietly doing far more work than it looks like on paper. See `examples/05_fibonacci_naive_recursion.c`, which prints a call counter so you can watch the blowup happen.

What happens without a base case

See `examples/06_missing_base_case_infinite_recursion.c` for the full story — in short: a recursive function with no base case (or a recursive case that never actually reaches it) calls itself forever. Each call adds another stack frame, and stack frames aren't infinite — eventually the program crashes with a **stack overflow**. That file shows the broken version *safely disabled* behind a comment (running it for real crashes the program on purpose, which isn't worth doing more than once), alongside the corrected version with a real base case, so you can see exactly what separates them.

Common beginner mistakes

- **No base case at all**, or a base case that's never actually reached — guaranteed infinite recursion and a crash.
- A recursive case that doesn't make **measurable progress** toward the base case (e.g. calling `countdown(n)` instead of `countdown(n - 1)` — identical arguments forever).
- Forgetting to **return** the recursive call's result when the function needs to hand a value back (`factorial(n - 1);` instead of `return n * factorial(n - 1);`; silently discards the computation).

- Assuming recursion is always "cleaner" — sometimes it quietly repeats huge amounts of work, as naive Fibonacci shows.
- Off-by-one base cases ($n == 1$ instead of $n == 0$, or vice versa) — trace a small example by hand before trusting a base case is right.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one — including the safely-disabled infinite recursion in 06, so you understand the danger without actually crashing anything you care about.
2. Complete **exercises/exercise1.c**.
3. Build the **module project**.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Module 10: recursion"  
git push
```

Next: **Module 11 — Dynamic Memory Allocation.**

Module 11 — Dynamic Memory Allocation

Hook: an array with no size guess anywhere in it

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int count;
    printf("How many scores do you want to enter? ");
    scanf("%d", &count);

    int *scores = malloc(sizeof(int) * count); // exactly `count` slots --
not a guess
    if (scores == NULL) {
        printf("malloc failed.\n");
        return 1;
    }

    for (int i = 0; i < count; i++) {
        printf("Score %d: ", i + 1);
        scanf("%d", &scores[i]);
    }

    int sum = 0;
    for (int i = 0; i < count; i++) {
        sum += scores[i];
    }
    printf("You entered %d scores, totaling %d.\n", count, sum);

    free(scores);
    return 0;
}
gcc hook.c -o hook
./hook
```

Run it and enter 3 when asked, followed by 3 scores — it works. Now run it again and enter 300, followed by 300 scores — it *still* works, with the exact same source code, no recompiling with a bigger number baked in. Back in Module 5 you had to pick an array size up front, like `int scores[100]`, and just hope it was big enough. This program has no such guess anywhere in it: it asks the heap for *exactly* the number of slots it needs, at the moment it finds out how many that is. By the end of this module you'll understand exactly how `malloc` does that, and you'll build a scoreboard that can even grow *after* it's already allocated.

Where variables actually live: stack vs. heap

Every "normal" variable you've written so far — including fixed-size arrays like `int scores[100]` — lives on the **stack**: the same per-function-call storage from Module 10's stack frames. Stack memory is fast and fully automatic — it appears when a function starts and disappears the instant that function returns — but its size has to be known at compile time, which is exactly why `int scores[100]` needs a fixed number in the brackets.

The **heap** is a separate, much larger pool of memory that isn't tied to any function call. Memory on the heap is allocated on request, *while your program is running*, and it stays reserved until you explicitly give it back — nothing frees it automatically just because a function returned. That trade means more responsibility (you must remember to free it — more on that below) in exchange for a size that isn't locked in at compile time. See `examples/01_stack_vs_heap_intro.c`.

Why fixed-size arrays fall short

`int scores[100]` forces a decision before you have any information: if only 3 students show up, 97 slots sit there wasted; if 150 show up, the array is simply too small and you're out of luck without editing the source and recompiling. Dynamic memory allocation lets you make that decision **at runtime**, based on real information like a `scanf` result — exactly what the hook just did.

malloc: asking the heap for memory

```
int *numbers = malloc(sizeof(int) * count);
```

`malloc(size)` asks the heap for a block of `size` **bytes** and returns a pointer to the start of that block. Two details matter every time you call it:

- **Always combine it with `sizeof`.** `malloc(count)` would only ask for `count` *bytes*, not `count` *ints* — `malloc(sizeof(int) * count)` asks for enough bytes to hold `count` *ints*, whatever an `int`'s actual size happens to be on this machine. Read this pattern as "enough room for `count` of this type."
- **Always check the result for `NULL` before using it.** If the heap can't satisfy the request (memory really can run out), `malloc` returns `NULL` instead of crashing outright. Using that `NULL` pointer as if it pointed to real memory — writing through it — is what actually crashes the program, and it does so with a much more confusing error than a clean, deliberate check would have given you.

```
if (numbers == NULL) {
    printf("Allocation failed.\n");
    return 1;
}
```

See `examples/02_malloc_and_free_basics.c`, which is the hook's pattern with the `free` half added in.

calloc: malloc that zero-initializes

```
int *counts = calloc(count, sizeof(int));
```

`calloc(count, size)` does almost the same job as `malloc`, with two differences: it takes the item count and item size as two separate arguments, and — this is the important part — **it guarantees every byte of the block starts at zero**. `malloc` makes no such promise; the memory it hands you contains whatever garbage values were already sitting there. Assuming a freshly `malloc`'d array starts at zero is a classic bug: it can look zeroed the first few times you run a program and then silently contain garbage later, or on a different machine. If you need zero-initialized memory, ask for it with `calloc` — don't hope for it from `malloc`. See `examples/03_calloc_vs_malloc.c`.

realloc: growing (or shrinking) a block you already have

```
int *temp = realloc(scores, sizeof(int) * newSize);
if (temp == NULL) {
    // scores is STILL valid here -- realloc left it untouched on failure
    free(scores);
    return 1;
}
scores = temp;
```

`realloc(pointer, newSize)` resizes a block you previously got from `malloc`, `calloc`, or `realloc` itself. It has two gotchas that cause real bugs if you miss them:

- **It can move the block.** If the memory right after your current block isn't free, `realloc` allocates a new block elsewhere, copies your old data into it, and returns a pointer to the new location. You must capture that return value — continuing to use the old pointer afterward reads or writes memory that may no longer belong to you.
- **On failure it returns `NULL` — and leaves the original block alone.** If you write `scores = realloc(scores, newSize);` and it fails, you've just overwritten your only pointer to the original block with `NULL`. That original memory is still allocated and still has your data in it, but you can no longer reach it to use it or to free it — a leak. Always `realloc` into a temporary pointer, check the temporary for `NULL`, and only then assign it back to your real pointer.

See `examples/04_realloc_growing_an_array.c`.

free: giving memory back

```
free(numbers);
numbers = NULL;
```

Heap memory is never returned automatically — you ask for it with `malloc/calloc/realloc`, and you're on the hook for giving it back with `free`. The rule is simple to state and easy to violate in a bigger program: **every successful `malloc`, `calloc`, or `realloc` needs exactly one matching `free`**. Not zero (that's a leak, next section). Not two (that's a double free, below). Exactly one.

Setting the pointer to `NULL` immediately after freeing it isn't required by the language, but it's a habit worth building now: it turns two dangerous, silent bugs into a harmless no-op and an obvious crash, covered next.

Memory leaks

A **memory leak** happens when you allocate a block and lose every pointer to it before calling `free` — usually because a function returns without freeing something it allocated, or a pointer gets overwritten before the block it pointed to was freed. The memory stays reserved for the rest of the program's run; nothing can ever reach it again to give it back. A leak doesn't crash your program on the spot, which is exactly what makes it dangerous — it quietly wastes more memory the longer a program runs, especially one that allocates repeatedly in a loop or across many function calls.

`examples/05_memory_leak_demonstration.c` shows the broken pattern (a function that mallocs and never frees) safely disabled in a comment, next to the fixed version that actually runs.

Dangling pointers and double frees

Two more dangers, both **undefined behavior** — not a clean, predictable error, but anything from silently wrong output to a crash to nothing visibly wrong at all until much later:

- A **dangling pointer** is a pointer whose memory has already been freed. `free(ptr)` releases the memory `ptr` points to, but it does *not* change `ptr` itself — the variable still holds that old address. Dereferencing it afterward (reading or writing through it) is a **use-after-free**: that memory may already have been handed to something else entirely.
- A **double free** is calling `free` twice on the same block. The heap's internal bookkeeping isn't designed to be told "here's this memory back" twice, and doing so corrupts that bookkeeping — undefined behavior again, and often not until much later in the program's run.

The fix for both is the habit from the previous section: **set a pointer to NULL immediately after freeing it**. `free(NULL)` is explicitly defined to do nothing, so an accidental second `free` becomes harmless, and dereferencing a NULL pointer crashes immediately and obviously instead of silently corrupting memory somewhere else. (A later module — Module 15 — introduces tools that can catch leaks and invalid memory access automatically; for now, the discipline in this module is what keeps you safe.)

See `examples/06_dangling_pointer_and_double_free.c`.

Common beginner mistakes

- **Forgetting to check `malloc/calloc/realloc` for NULL** before using the result — the single most common bug in this module.
- **Assuming `malloc`'d memory starts at zero**. It doesn't; use `calloc` if you need that guarantee.
- **Losing the original pointer on `realloc` failure** by writing `ptr = realloc(ptr, newSize);` directly instead of through a temporary pointer — this leaks the original block if `realloc` ever fails.
- **Mismatched allocation and freeing** — freeing something twice, never freeing it at all, or freeing a pointer that was never returned by `malloc/calloc/realloc` in the first place.
- **Using a pointer after freeing it** without setting it to NULL first — a dangling pointer that looks like it might still work, until it doesn't.
- **Forgetting the `sizeof(type) * count` pattern** and allocating the wrong number of *bytes* instead of the wrong number of *items*.

Try it yourself

1. Work through every file in `examples/`, compiling and running each one — including the safely-disabled leak and dangling-pointer bugs in 05 and 06, so you understand the danger without actually corrupting anything.
2. Complete `exercises/exercise1.c`.
3. Build the **module project**.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 11: dynamic memory allocation"
git push
```

Next: **Module 12 — Linked Lists**.

Module 12 — Linked Lists

Hook: a treasure hunt made of pointers

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    char clue[40];
    struct Node *next;
} Node;

int main(void) {
    // Build the treasure hunt by hand: four clues, each one pointing
    // to the next.
    Node *stop1 = malloc(sizeof(Node));
    Node *stop2 = malloc(sizeof(Node));
    Node *stop3 = malloc(sizeof(Node));
    Node *stop4 = malloc(sizeof(Node));

    snprintf(stop1->clue, sizeof(stop1->clue), "Look under the old oak tree");
    snprintf(stop2->clue, sizeof(stop2->clue), "Check behind the waterfall");
    snprintf(stop3->clue, sizeof(stop3->clue), "Search the lighthouse steps");
    snprintf(stop4->clue, sizeof(stop4->clue), "You found the treasure!");

    stop1->next = stop2;
    stop2->next = stop3;
    stop3->next = stop4;
    stop4->next = NULL;    // the last clue points nowhere -- hunt over

    Node *start = stop1;    // "start" is the only thing that remembers stop1

    Node *current = start;
    while (current != NULL) {
        printf("%s\n", current->clue);
        current = current->next;
    }

    free(stop1);
    free(stop2);
    free(stop3);
    free(stop4);
    return 0;
}
gcc hook.c -o hook
./hook
```

Four separate blocks of memory, allocated one at a time, each holding a clue **and directions to the next one** — nothing but pointers holds this chain together. `start` doesn't know about `stop2`, `stop3`, or `stop4` directly; it only knows `stop1`, and `stop1` knows where `stop2` is, and so on. Follow the chain far enough and you reach `stop4`, whose `next` is `NULL` — the treasure, and the end of the hunt. That chain is a **linked list**, and by the end of this module you'll be adding stops to it, removing them, and cleaning the whole thing up without losing a single node.

Why not just use a bigger array?

Module 11 gave you `realloc` — an array that can grow. That solves "I don't know how many elements I'll need," but it doesn't solve everything. Picture removing the 3rd element out of 10,000 from an array: elements 4 through 10,000 all have to physically shift one slot to the left to close the gap. Insert something in the middle and everything after it has to shift the other way to make room. For **frequent insertions and deletions in the middle**, that shifting adds up fast.

A linked list solves this differently: instead of one big contiguous block, each element is its own small chunk of memory, and elements point to their neighbors instead of sitting next to them. Removing an element in the middle becomes "change where two pointers point" — nothing shifts. The trade-off: you lose the array's instant `arr[500]`-style access — to reach the 500th node you have to walk there one `next` at a time. Arrays and linked lists are good at opposite things; you'll pick whichever one fits the job.

What is a node?

A **node** is a struct that holds data **plus a pointer to another node of the same type**. That pointer-to-its-own-type is called a **self-referential struct**, and it's the one new idea this whole module is built on:

```
typedef struct Node {
    int value;
    struct Node *next;    // a pointer to another Node
} Node;
```

Read that `next` line carefully — it says `struct Node *next`, not `Node *next`. This trips up almost everyone the first time. The `typedef` on the first line says "once this whole declaration is finished, you may call this type `Node`" — but that name doesn't exist yet while you're still in the middle of defining the struct's own body. C already has a name you can use at that point, though: `struct Node`, from `typedef struct Node {`. So inside the braces, before the `typedef` is complete, you must write `struct Node *next` — after the closing `} Node;`, everywhere else in your code, you can go back to using the short name `Node`. See [examples/01_what_is_a_node.c](#).

Building a list by hand

A list is just nodes, `malloc`'d one at a time and linked together with plain assignments:

```
Node *node1 = malloc(sizeof(Node));
Node *node2 = malloc(sizeof(Node));
Node *node3 = malloc(sizeof(Node));

node1->value = 10;
node2->value = 20;
node3->value = 30;

node1->next = node2;    // node1 points to node2
node2->next = node3;    // node2 points to node3
node3->next = NULL;     // node3 is last -- nothing after it

Node *head = node1;    // head remembers where the list starts
```

`head` is the only variable in this whole program that remembers where `node1` lives. Everything else — `node2`, `node3`, the rest of the list — is only reachable **by following `next` pointers starting from `head`**. Hold onto that idea; it matters a lot later in this module. See [examples/02_building_a_list_by_hand.c](#).

Traversal: walking the list

To do anything with a list — print it, search it, count it — you walk it with a separate pointer, conventionally called `current`, so you never lose track of `head`:

```
Node *current = head;
while (current != NULL) {
    printf("%d -> ", current->value);
    current = current->next;
}
printf("NULL\n");
```

`current` starts at `head` and moves one node at a time (`current = current->next`) until it falls off the end of the list — which is exactly when it becomes `NULL`. That `while (current != NULL)` check is the heartbeat of almost every linked list function you'll write. See

[examples/03_traversing_a_list.c](#).

Inserting at the head

The cheapest possible insertion: point the new node at the current head, then make the new node the head.

```
Node *insertAtHead(Node *head, int value) {
    Node *newNode = malloc(sizeof(Node));
    if (newNode == NULL) {
        return head;    // allocation failed -- list unchanged
    }
    newNode->value = value;
    newNode->next = head;    // new node points at the OLD head first...
    return newNode;        // ...then becomes the new head
}
```

Order matters: `newNode->next = head;` has to happen **before** `head` is overwritten, or you'd lose the rest of the list. That's why the function returns the new head instead of modifying `head` in place — the caller has to remember to do `head = insertAtHead(head, ...)`. See

[examples/04_inserting_at_the_head.c](#).

Inserting and deleting anywhere else

Inserting or deleting in the *middle* means walking to the node just **before** the spot you care about, then relinking pointers around it.

Inserting after a target node:

```
newNode->next = current->next;    // new node points to what used to be next
current->next = newNode;          // the node before it now points to new node
```

Deleting a node (not the head) needs a *previous* pointer alongside `current`, so you can skip over the node being removed:

```
previous->next = current->next;    // skip over the node being removed
free(current);
```

Deleting the **head** is a special case, because there's no previous node to relink — you have to update `head` itself instead:

```

if (head->value == value) {
    Node *doomed = head;
    head = head->next;
    free(doomed);
    return head;
}

```

See `examples/05_inserting_and_deleting_anywhere.c` for both operations working together on the same list, insertions and deletions verified against the expected list contents at each step.

Freeing the whole list

Every node was `malloc`'d individually, so every node has to be `free`'d individually — Module 11's rule about matching every `malloc` still applies here, just once per node instead of once total. The tempting but broken approach:

```

// BROKEN -- do not do this:
Node *current = head;
while (current != NULL) {
    free(current);
    current = current->next;    // current was JUST freed -- reading
                                // current->next here is undefined
                                // behavior, often a crash
}

```

The fix is one line of reordering: **save next before you free the current node**, so you're never reading through memory you just returned to the system.

```

Node *current = head;
while (current != NULL) {
    Node *next = current->next;    // save it first
    free(current);                // now safe to free current
    current = next;               // advance using the SAVED pointer
}

```

See `examples/06_freeing_the_whole_list.c`.

Don't lose the head

`head` is the single pointer your whole program uses to reach every node in the list. If you ever overwrite it without freeing (or otherwise saving) the list it used to point to —

```

head = someBrandNewNode;    // the old list is now unreachable!

```

— every node the old `head` used to lead to is still sitting in memory, still allocated, but there is no longer any pointer anywhere in your program that remembers where any of them are. That's a **memory leak**, and it's permanent for the life of the program: you can't `free` what you can no longer reach. Always free (or deliberately hand off to another pointer) a list's nodes before its `head` pointer gets reassigned to something else.

Common beginner mistakes

- Writing `Node *next;` instead of `struct Node *next;` inside the struct body — the typedef name doesn't exist yet at that point.
- Freeing a node and then reading `current->next` afterward — always save `next` into a local variable *before* calling `free`.

- Forgetting that insertion/deletion functions must **return** the (possibly new) head, and forgetting to write `head = theFunction(...)` at the call site — without it, changes to the head are silently lost.
- Overwriting `head` (or any pointer that's the only reference to part of a list) without freeing what it used to point to — an instant, unrecoverable leak.
- Deleting or inserting in the middle without keeping a `previous` pointer — you need the node *before* the target to relink around it.
- Forgetting the `NULL` check after `malloc` — a failed allocation returns `NULL`, and dereferencing it crashes the program (Module 11).

Try it yourself

1. Work through every file in **examples/**, compiling and running each one.
2. Complete **exercises/exercisel.c**.
3. Build the **module project**.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Module 12: linked lists"  
git push
```

Next: **Module 13 — Stacks & Queues.**

Module 13 — Stacks & Queues

Hook: building your own Ctrl+Z

Compile and run this exactly as written — don't read ahead first:

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    const char *action;
    struct Node *next;
} Node;

Node* push(Node *top, const char *action) {
    Node *newNode = malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("malloc failed -- history unchanged.\n");
        return top;
    }
    newNode->action = action;
    newNode->next = top;
    return newNode;
}

Node* pop(Node *top) {
    if (top == NULL) {
        printf("Nothing left to undo.\n");
        return NULL;
    }
    printf("Undoing: %s\n", top->action);
    Node *oldTop = top;
    top = top->next;
    free(oldTop);
    return top;
}

int main(void) {
    Node *history = NULL;

    history = push(history, "Typed 'Hello'");
    printf("Did: Typed 'Hello'\n");
    history = push(history, "Typed ', world!'");
    printf("Did: Typed ', world!'\n");
    history = push(history, "Deleted a paragraph");
    printf("Did: Deleted a paragraph\n");

    printf("\nPressing Ctrl+Z three times:\n");
    history = pop(history);
    history = pop(history);
    history = pop(history);

    return 0;
}
gcc hook.c -o hook
./hook

```

Three actions go in, and three actions come back out in exactly the **reverse** order — the last thing you did is the first thing undone. That's not a coincidence; it's the entire idea behind this module, already working. By the end of this module you'll understand exactly why `push/pop` naturally produce that reversal, you'll meet its opposite (a structure where the *first* thing in is the first thing out), and you'll build a small interactive text editor with real undo in this module's project. See [examples/04_undo_simulation_with_a_stack.c](#) for this exact program.

LIFO vs. FIFO: two disciplines for ordering data

A **stack** and a **queue** are both just linked lists (Module 12) — the *only* difference is a rule about which end you're allowed to add to and remove from:

- A **stack** is **LIFO** — Last In, First Out. Whatever you added most recently is the first thing that comes back out. Think of a stack of plates: you add to the top, and you take from the top. That's exactly what the hook just demonstrated — `push` added to one end, and `pop` removed from that *same* end, which is why the order came back reversed.
- A **queue** is **FIFO** — First In, First Out. Whatever you added *first* is the first thing that comes back out. Think of a line at a checkout counter: you join at the back, and you're served from the front. Add at one end, remove from the *other* end, and the order comes back the same as it went in.

Everything else in this module is really just: how do you build each of these on top of a linked list, and why does a queue need a little more bookkeeping than a stack does?

Implementing a stack with a linked list

A stack needs exactly two operations: **push** (add a new item) and **pop** (remove and return the most recently added item). If you did Module 12's insert-at-head exercise, you've already written half of this — pushing onto a stack IS inserting a new node at the head of a linked list:

```
typedef struct Node {
    int value;
    struct Node *next;
} Node;

Node* push(Node *top, int value) {
    Node *newNode = malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("malloc failed -- stack unchanged.\n");
        return top;
    }
    newNode->value = value;
    newNode->next = top;
    return newNode;
}
```

We call the head pointer `top` here instead of `head` — same pointer, different name, because stack terminology talks about the "top" of the stack rather than the "front" of a list. `pop` is remove-the-head, with one rule that matters more here than it ever did for a plain linked list: **always check for an empty stack before popping**.

```
Node* pop(Node *top, int *poppedValue) {
    if (top == NULL) {                // ALWAYS check this first
        printf("Stack is empty -- cannot pop.\n");
        return NULL;
    }
    Node *oldTop = top;
    *poppedValue = top->value;
    top = top->next;
    free(oldTop);
    return top;
}
```

Skipping that check means dereferencing `top->value` when `top` is `NULL` — an instant crash. Because `push` and `pop` only ever touch the head, neither one has to walk the list, so both run in constant time no matter how large the stack gets. See `examples/01_stack_push_and_pop.c`.

A simpler alternative: array-based stacks

A linked-list stack can grow without limit — every `push` just mallocs one more node. But sometimes you know in advance you'll never need more than, say, 100 items, and in that case a plain array is a simpler alternative: no `malloc`, no `free`, no pointers to chase, just an array and an integer tracking the index of the top element (`-1` means empty).

The tradeoff is the mirror image of the linked-list version: **simpler code, but a fixed maximum size decided at compile time** — push past that limit and you have to explicitly refuse, instead of just allocating another node. Neither approach is "better" in general; pick the one that matches what you actually know about your data ahead of time. See `examples/02_array_based_stack_alternative.c`.

Implementing a queue with a linked list

A queue needs **enqueue** (add to the back of the line) and **dequeue** (remove from the front). This is where a queue genuinely needs more than a stack does: enqueue adds at the **tail**, dequeue removes from the **head** — two different ends.

If you only tracked a head pointer, `enqueue` would have to walk the *entire* list every single time, just to find the last node to attach the new one to — that's an $O(n)$ operation that gets slower as the queue grows, for something that should be instant. The fix is to track a **tail pointer too**, updated on every enqueue, so you always know exactly where the last node is without looking:

```

typedef struct {
    Node *head;    // front of the line -- dequeue removes from here
    Node *tail;    // back of the line -- enqueue adds here
} Queue;

void enqueue(Queue *q, int value) {
    Node *newNode = malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("malloc failed -- queue unchanged.\n");
        return;
    }
    newNode->value = value;
    newNode->next = NULL;

    if (q->tail == NULL) {           // queue was empty
        q->head = newNode;
        q->tail = newNode;
    } else {
        q->tail->next = newNode;
        q->tail = newNode;
    }
}

```

Because enqueue and dequeue each need to update two pointers at once (head and tail), we pass a **pointer to the whole Queue struct** (Module 7/8's `->` operator) instead of returning a new value the way push did for the single-pointer stack. dequeue has the same empty-check discipline as pop — plus one extra rule:

```

int dequeue(Queue *q, int *value) {
    if (q->head == NULL) {           // ALWAYS check this first
        printf("Queue is empty -- cannot dequeue.\n");
        return 0;
    }
    Node *oldHead = q->head;
    *value = oldHead->value;
    q->head = oldHead->next;

    if (q->head == NULL) {           // that was the last node --
        q->tail = NULL;              // the tail must be reset too
    }

    free(oldHead);
    return 1;
}

```

That last `if` matters: after removing the only node in the queue, head correctly becomes NULL on its own, but tail does **not** — it's still pointing at the node you just freed unless you reset it yourself. Forgetting this leaves tail **dangling** (pointing at freed memory), and the next enqueue would write through it and corrupt the heap. See `examples/03_queue_enqueue_and_dequeue.c`.

Real-world use cases

- **Stacks power undo history.** Every "undo" button you've ever used — in a text editor, an image editor, your browser's back button — is LIFO by nature: the most recent action is always the first one reversed. See `examples/04_undo_simulation_with_a_stack.c`, which is the hook above.

- **Queues power task and print queues.** When several jobs are requested and handled one at a time, fairness usually means FIFO: whoever asked first gets served first. See `examples/05_task_queue_simulation.c`, a small print-queue simulation.

Common beginner mistakes

- **Popping or dequeuing without checking for empty first** — the single most common bug in this module. `top == NULL` or `q->head == NULL` must be checked *before* touching `->value`.
- **Mixing up which end a queue operates on** — enqueue at the tail, dequeue at the head. Doing both at the same end turns your "queue" into a stack by accident.
- **Forgetting to reset `tail` to `NULL`** when `dequeue` removes the last node, leaving it dangling and corrupting the next `enqueue`.
- **Forgetting to free a popped/dequeued node** — same rule as Module 11 and 12: every node that came from `malloc` needs exactly one matching `free`, whether it leaves the structure through `pop`, `dequeue`, or a full cleanup loop at the end of the program.
- **Assuming a stack and a queue are "the same thing with different names."** The node struct is identical; the *rule about which end you touch* is the entire difference, and it's what makes one LIFO and the other FIFO.

Try it yourself

1. Work through every file in `examples/`, compiling and running each one.
2. Complete `exercises/exercisel.c`.
3. Build the **module project**.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 13: stacks and queues"
git push
```

Next: **Module 14 — Multi-File Programs.**

Module 14 — Multi-File Programs (& the Preprocessor)

Hook: the same program, in two pieces

Compile and run both of these exactly as written — don't read ahead first.

Version 1 — one file, everything in it: `examples/01_single_file_before/main.c`

```
gcc examples/01_single_file_before/main.c -o before
./before
```

Version 2 — the exact same program, split into pieces:

`examples/02_split_into_header_and_source/` contains `main.c`, `mathreport.c`, and `mathreport.h`.

```
gcc examples/02_split_into_header_and_source/main.c \
    examples/02_split_into_header_and_source/mathreport.c \
    -o after
./after
```

Same output, both times, character for character. The only thing that changed is *where* the code lives: one `gcc` command that now lists two `.c` files instead of one, a new `.h` file describing what's available, and a `main.c` that's shrunk down to just "what does this program do," with "how" moved somewhere else. That's the whole subject of this module: **splitting a program across files without changing what it does** — so it's something you could hand to a teammate to work on independently, or just find again in six months without scrolling.

Why one file stops scaling

Every program you've written so far has fit in a single `.c` file comfortably. That stops being true as programs grow. Picture a 400-line `.c` file: a few structs, a dozen functions for validating input, computing statistics, formatting output, saving to a file — all of it scrolling past in one window. Now imagine you need to fix one bug in "the function that formats a report." You know it's *in there somewhere* between line 40 and line 380. That search — scrolling, `Ctrl+F`-ing, losing your place — is the actual, physical cost of a program outgrowing one file. It also means:

- Two people can't easily work on the same program at once — you're both editing the same file and fighting over the same lines.
- Code you wrote for one program (a temperature converter, say) can't be reused in another program without copy-pasting the whole file and deleting what you don't need.
- Everything recompiles every time, even if you only changed one small function.

The fix isn't a new language feature — it's organization. **Header files** and **source files**, working together, are how C code gets organized once it outgrows a single file.

Splitting a program: headers declare, source files define

A C program split across files always follows the same shape:

- A **header file** (`.h`) holds **declarations**: function prototypes, struct definitions, and `#define` constants. It's the *menu* — what this piece of code offers, with no detail about how any of it works.
- A **source file** (`.c`) holds **definitions**: the actual function bodies. It's the *kitchen* — how the menu items actually get made.

Look at `examples/02_split_into_header_and_source/mathreport.h`:

```
#ifndef MATHREPORT_H
#define MATHREPORT_H

#define SEPARATOR_WIDTH 20

int square(int n);
int cube(int n);
void printSeparator(void);
void printReport(int n);

#endif
```

Every line is a *promise*, not an implementation — "somewhere, a function called `square` exists that takes an `int` and returns an `int`." The actual bodies live in `mathreport.c`, which `#includes` the header and then defines every function it promised. Any file that wants to use `square()` just needs `#include "mathreport.h"` — it doesn't need to see, or even care about, how `square` is implemented.

`#include "..."` VS. `#include <...>`

You've used `#include <stdio.h>` since Module 1. Now compare it with `#include "mathreport.h"` in `examples/02_split_into_header_and_source/main.c`:

- **Angle brackets** (`<stdio.h>`) — for **system/standard library** headers that shipped with your compiler. The compiler looks in its own standard install locations.
- **Double quotes** (`"mathreport.h"`) — for **your own local files**. The compiler looks in your project directory first (starting next to the file doing the including), and only falls back to the system locations if it doesn't find a match there.

Practically: if you wrote the header yourself and it lives in your project, use double quotes. If it's part of the C standard library, use angle brackets. Mixing them up (`#include <mathreport.h>`) usually produces a "file not found" error, because the compiler never thinks to look in your project folder.

Include guards: why every header needs one

Try this on purpose. `examples/03_include_guards_demo/no_guards/` has a `point.h` with **no** include guard, included by both `main.c` *and* `shape.h` — and `main.c` includes `shape.h` too. Compile it:

```
gcc examples/03_include_guards_demo/no_guards/main.c \
    examples/03_include_guards_demo/no_guards/point.c \
    examples/03_include_guards_demo/no_guards/shape.c \
    -o broken
```

That fails with something like `error: redefinition of 'struct Point'`. Here's why: `#include` is a **preprocessor** directive — before the compiler ever looks at your code, the preprocessor literally copy-pastes the entire contents of the included file in, textually, replacing the `#include` line. `main.c` includes `point.h` directly *and* includes `shape.h`, which **also** includes `point.h` internally. Once the preprocessor is done, `main.c`'s expanded text contains the entire `struct Point` definition **twice** — and the compiler, quite reasonably, refuses to accept the same `struct` defined twice in one file.

The fix is an **include guard**, wrapped around everything in a header:

```
#ifndef POINT_H
#define POINT_H

// ... the header's actual contents ...

#endif
```

The first time `point.h` gets pulled into a file, `POINT_H` isn't defined yet, so the preprocessor defines it and keeps the contents. Every *later* `#include "point.h"` in that same file finds `POINT_H` already defined and skips straight past everything to `#endif` — the struct only ever gets pasted in once. See **examples/03_include_guards_demo/with_guards/**, which is the identical program with only that one fix, and compiles and runs cleanly:

```
gcc examples/03_include_guards_demo/with_guards/main.c \
    examples/03_include_guards_demo/with_guards/point.c \
    examples/03_include_guards_demo/with_guards/shape.c \
    -o fixed
./fixed
```

Every header you write from now on should have an include guard. The convention is simple and consistent: `#ifndef` using the filename in capitals with underscores, matching `#define` right below it, `#endif` as the very last line.

#define VS. const

You met `const` back in Module 3, and you've now used `#define` for `SEPARATOR_WIDTH` and the include-guard names above. They can look similar but work completely differently:

```
#define MAX_STUDENTS 30    // preprocessor: pure text substitution,
                          // happens BEFORE compilation even starts

const int maxStudents = 30; // compiler: a real, typed variable that
                          // happens to be read-only
```

`#define` isn't a variable at all — the preprocessor just replaces every later appearance of `MAX_STUDENTS` with the literal text `30`, the same mechanical find-and-replace it uses to expand `#include`. `const int` is a genuine variable, with a type the compiler checks, that simply can't be reassigned after it's initialized.

For a beginner, the practical rule: **prefer `const` for values** — you get type checking and it behaves like every other variable you already know. Reach for `#define` mainly for two things this module actually needs it for: include-guard names (which must be pure preprocessor text, not a real variable) and small constants you want shared identically across several files via a header, like `SEPARATOR_WIDTH` above.

Compiling multiple .c files together

`gcc` needs to know about **every** `.c` file that contributes code to your program — headers are never listed on the command line, only `.c` files:

```
gcc main.c helpers.c -o program
```

Miss one (say you only typed `gcc main.c -o program` when `main.c` calls a function defined in `helpers.c`) and you'll get a **linker error** like undefined reference to `'someFunction'` — the compiler understood *that* the function exists (it saw the prototype in the header you included), but

nothing ever supplied its actual body, because `helpers.c` was never compiled in. The fix is almost always the same: check your `gcc` command lists every `.c` file the program needs.

A minimal Makefile

Retyping a long `gcc` command with several `.c` files every time you test a change gets old fast. A **Makefile** fixes that by giving the command a name. See

examples/04_compiling_with_a_makefile/Makefile:

```
all:
    gcc main.c stats.c -o program

clean:
    rm -f program
```

`all` and `clean` are **targets** — names for a recipe. Typing `make` in that folder runs the recipe under the first target (`all`) by default; `make clean` runs the recipe under `clean`. The one rule that trips everyone up once: **the indented line under a target must start with a real tab character**, not spaces — most editors handle this automatically inside a file literally named `Makefile`, but it's worth knowing if `make` ever complains about a "missing separator."

```
cd examples/04_compiling_with_a_makefile
make
./program
```

This is intentionally the smallest useful Makefile, not a real build system — enough to stop retyping one command, nothing more.

Common beginner mistakes

- **Forgetting the include guard** — see **Include guards** above; this is the single most common multi-file bug.
- **Putting a function body in a header instead of just a prototype** — works by accident in a tiny one-file-includes-it-once program, then breaks the moment two `.c` files both include that header: each `.c` file gets its own copy of the function, and the linker refuses to build with the same function defined twice.
- **Defining a variable (not just a prototype) in a header** — the same problem as above, one level worse: `int total = 0;` sitting directly in a header, included by two different `.c` files, gives *each* `.c` file its own separate copy of `total`. The linker then reports `total` as multiply defined and refuses to build. Declarations (prototypes) belong in headers; the actual variable belongs in exactly one `.c` file.
- **Mixing up "`...`" and `<...>` in `#include`** — your own files use double quotes; standard library headers use angle brackets.
- **Forgetting to list every `.c` file** on the `gcc` command line, then being confused by an `undefined reference` linker error.
- **A Makefile recipe line indented with spaces instead of a tab** — `make` requires a literal tab character before every recipe line.

Try it yourself

1. Work through every folder in **examples/**, compiling and running each one — including deliberately reproducing (and then fixing) the duplicate-definition error in `03_include_guards_demo/`.
2. Complete **exercises/exercisel.c** — split a single-file program into a header and source pair yourself.
3. Build the **module project**.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Module 14: multi-file programs"  
git push
```

Next: **Module 15 — Debugging & Memory Safety.**

Module 15 — Debugging & Memory Safety

You've been writing programs with `malloc`, pointers, and linked structures for a few modules now, which means you've also been writing the occasional crash and the occasional silent memory leak -- everyone does. This module doesn't teach you to stop making those mistakes entirely (nobody does that). It teaches you to find them in minutes instead of hours, using the same tools working C programmers reach for every day.

Hook: the crash is not a mystery, it just looks like one

Compile and run this exactly as written -- don't read ahead first:

```
#include <stdio.h>

void printScore(int *score) {
    printf("Score: %d\n", *score);
}

int main(void) {
    int *currentScore = NULL;

    printScore(currentScore);

    printf("This line never runs.\n");
    return 0;
}
gcc -g hook.c -o hook
./hook
Segmentation fault
```

That's it. That's the entire error message. No line number, no function name, no hint about `currentScore` being `NULL`. If this were a 2,000-line program instead of nine lines, you'd have almost nothing to go on.

Now run the *exact same program* under a debugger instead:

```
$ gdb ./hook
(gdb) run
Starting program: /home/you/15-debugging-and-memory-safety/hook

Program received signal SIGSEGV, Segmentation fault.
0x0000000000401156 in printScore (score=0x0) at hook.c:4
4      printf("Score: %d\n", *score);
(gdb) bt
#0  0x0000000000401156 in printScore (score=0x0) at hook.c:4
#1  0x0000000000401189 in main () at hook.c:11
```

Same crash, same program -- but now you know the exact line (`hook.c:4`), the exact function (`printScore`), and the exact reason (`score=0x0`, i.e. `score` is `NULL`), in about five seconds. The crash was never mysterious. You just needed the right tool to ask the program what happened. That's this whole module.

A quick note on your platform

`gdb` and `valgrind` both work great on Linux and on Windows via WSL. `gdb` is also generally fine on Intel Macs (it may need a one-time setup step to grant it permission to control other processes).

`valgrind`, however, **does not reliably support Apple Silicon (M1/M2/M3/M4) Macs** -- which a good number of you are probably using right now. This is a well-known, common situation, not something broken about your setup.

If `valgrind` isn't available on your machine, you can still do this whole module:

- Work through every `gdb` section normally -- it's unaffected.
- For the `valgrind` sections, either use a Linux machine or WSL if you have access to one, **or** treat the example output shown below as what you're learning to read -- the skill is interpreting a `valgrind` report, and you can build that skill from real examples even before you've generated your own.
- You'll get hands-on `valgrind` practice in the Part 2 capstone if your team works on a Linux/WSL machine, or you'll do a careful manual `malloc/free` audit instead if not -- Module 19 explains both paths.

Nothing below assumes you have `valgrind` running in front of you right now. Read the reports, understand what they're telling you, and you're learning the real skill either way.

What actually happens during a segfault

A **segmentation fault** ("segfault") happens when your program tries to read or write memory it doesn't have permission to touch -- most often through a `NULL` pointer (like the hook) or a pointer that never held a valid address in the first place (an uninitialized pointer, or one that's already been freed).

Your program doesn't own all of memory; the operating system hands it specific regions to use. The instant your program reaches outside those regions, the OS steps in and kills the process immediately, before anything worse can happen (like corrupting some *other* program's memory). That abrupt kill is the segfault. It's not your program "breaking" in some soft, recoverable way -- it's the OS actively stopping it.

This is exactly why a segfault gives you almost no information by default: the OS's job is to stop the damage, not to explain it to you. That's the debugger's job.

Compiling with debug symbols: `-g`

```
gcc -g file.c -o program
```

The `-g` flag tells `gcc` to bake extra information into the compiled program: which machine instruction corresponds to which line of *your* source file, and what your variables are actually named. Without `-g`, a debugger can only show you raw memory addresses -- technically correct, practically useless. With `-g`, it can show you `hook.c:4` and `score=0x0` instead.

`-g` doesn't change how your program runs or what it prints -- it only adds information *for tools like `gdb`* to use. Get in the habit of compiling with it any time you're about to debug something:

```
gcc -g -Wall file.c -o program
```

Using gdb: the small, practical subset

gdb (the GNU Debugger) lets you run your program under close supervision -- pausing it, stepping through it one line at a time, and inspecting variables as it goes. It has an enormous command set; you need about six commands to handle most beginner debugging.

Start it on a program compiled with `-g`:

```
gdb ./program
```

Running a program that crashes. Type `run` (or `r`) at the `(gdb)` prompt. The program executes normally until it crashes, then `gdb` pauses it *at the exact instant of the crash* instead of letting the OS kill it silently:

```
(gdb) run
Starting program: /path/to/program

Program received signal SIGSEGV, Segmentation fault.
0x0000000000401156 in printScore (score=0x0) at hook.c:4
4      printf("Score: %d\n", *score);
```

Reading a backtrace. Once stopped, `bt` (backtrace) prints the chain of function calls that got you there, most recent call first:

```
(gdb) bt
#0  0x0000000000401156 in printScore (score=0x0) at hook.c:4
#1  0x0000000000401189 in main () at hook.c:11
```

#0 is where the crash happened; #1 is who called that function; and so on outward. For a crash, #0 is almost always where you want to start looking.

Setting a breakpoint. Sometimes you don't want to wait for a crash -- you want to pause at a specific spot and watch what happens from there. `break` (or `b`) sets that pause point, either by function name or by file and line number:

```
(gdb) break printScore
(gdb) break hook.c:10
```

Then `run` starts the program and it stops automatically the moment it reaches that breakpoint, before executing that line.

Moving through code. Once stopped at a breakpoint:

- `next` (or `n`) runs the current line and moves to the next one, stepping over function calls (they run to completion without you following into them).
- `step` (or `s`) does the same, but steps *into* a function call if the current line makes one -- useful when the bug might be inside that call.

Inspecting a value. `print` (or `p`) followed by a variable name shows its current value, at any point while the program is paused:

```
(gdb) print currentScore
$1 = (int *) 0x0
```

`0x0` is `NULL` -- there's your bug, confirmed directly from the running program instead of inferred from a crash message.

Type `quit` (or `q`) to leave `gdb` when you're done.

That's the whole practical toolkit: `run`, `bt`, `break`, `next/step`, `print`, `quit`. Everything else `gdb` can do, you can look up when you actually need it.

(The exact addresses and version banner `gdb` prints will differ on your machine -- what matters is the function name, file, and line number, which will match.)

Using `valgrind`: finding leaks and invalid memory access

`gdb` is for crashes -- things that stop your program. `valgrind` is for the bugs that **don't** stop your program: memory that's silently leaked, or memory that's read or written outside the bounds you allocated. Both can run for a long time looking completely fine before they cause real damage.

Run it exactly like your program, just with `valgrind` in front:

```
gcc -g file.c -o program
valgrind ./program
```

Reading a "definitely lost" leak report

Take `examples/02_memory_leak_and_valgrind.c`, which allocates a small buffer inside `printReport()` and never frees it. Running it under `valgrind` produces a report shaped like this (*the example below is reconstructed from a real `valgrind` run, since this course's build environment doesn't have `valgrind` installed to capture live output from -- the shape, wording, and line numbers are exactly what your own run against this file will show if you have `valgrind` available*):

```
==12345== Memcheck, a memory error detector
==12345== Command: ./leak_demo
==12345==
Doubled scores:
    20
    40
    60
    80
Report printed -- but we just leaked memory doing it.
==12345==
==12345== HEAP SUMMARY:
==12345==    in use at exit: 16 bytes in 1 blocks
==12345== total heap usage: 1 allocs, 0 frees, 16 bytes allocated
==12345==
==12345== 16 bytes in 1 blocks are definitely lost in loss record 1 of 1
==12345==    at 0x4848899: malloc (vg_replace_malloc.c:381)
==12345==    by 0x1091A4: printReport (02_memory_leak_and_valgrind.c:18)
==12345==    by 0x1091FD: main (02_memory_leak_and_valgrind.c:39)
==12345==
==12345== LEAK SUMMARY:
==12345==    definitely lost: 16 bytes in 1 blocks
==12345==    indirectly lost: 0 bytes in 0 blocks
==12345==    possibly lost: 0 bytes in 0 blocks
==12345==    still reachable: 0 bytes in 0 blocks
==12345== ERROR SUMMARY: 1 errors from 1 contexts
```

Three things to read, in order:

1. **The program's own output still prints normally** -- `valgrind` doesn't stop your program, it watches it. A leak never crashes anything, which is exactly why you need a tool to catch it.
2. **definitely lost: 16 bytes in 1 blocks** -- this is the headline. "Definitely lost" means memory that's gone forever: nothing in your program still holds a pointer to it, so it can

never be freed. 16 bytes is `sizeof(int) * 4` -- exactly the one allocation that never got a matching `free()`.

3. **The backtrace under "definitely lost"** -- reads the same direction as `gdb`'s `malloc` was called from `printReport` at line 18, which was called from `main` at line 39. That's the exact allocation to go add a `free()` for.

Reading an "invalid read" report

Take `examples/03_invalid_read_and_valgrind.c`, which allocates room for 5 `ints` and then reads index 5 -- one past the end. Ordinary execution won't crash (you tested this yourself if you ran the file), which is exactly the danger: it just silently reads whatever happens to be in adjacent memory. `valgrind` catches it directly (*again, reconstructed to match a real run against this exact file, valid indices and all*):

```
==12346== Memcheck, a memory error detector
==12346== Command: ./invalid_read_demo
==12346==
Last valid value: 500
==12346== Invalid read of size 4
==12346==    at 0x109184: main (03_invalid_read_and_valgrind.c:30)
==12346== Address 0x4a4b054 is 0 bytes after a block of size 20 alloc'd
==12346==    at 0x4848899: malloc (vg_replace_malloc.c:381)
==12346== by 0x109160: main (03_invalid_read_and_valgrind.c:20)
==12346==
One past the end:  0
==12346==
==12346== HEAP SUMMARY:
==12346==    in use at exit: 0 bytes in 0 blocks
==12346== total heap usage: 1 allocs, 1 frees, 20 bytes allocated
==12346== ERROR SUMMARY: 1 errors from 1 contexts
```

Read it the same way: **Invalid read of size 4** at line 30 is the headline -- a 4-byte (one `int`) read that shouldn't have happened. Underneath, `valgrind` tells you exactly *why*: address `0x4a4b054` is "0 bytes after a block of size 20" that was allocated at line 20 -- in other words, precisely one element past the 5-`int` (20-byte) array you asked for. No guessing required: the report names the exact allocation and the exact overstep.

Defensive programming checklist

Most of what this module diagnoses is preventable with four habits.

`examples/04_defensive_programming_checklist.c` shows all four working together in one file:

- **Always initialize pointers** -- to `NULL` if you don't have a real address for them yet. A `NULL` pointer that gets misused crashes immediately and obviously; an *uninitialized* pointer can point anywhere, including memory that happens to "work" by pure luck until it doesn't.
- **Always check `malloc/fopen` return values.** Both return `NULL` on failure instead of crashing on the spot -- if you don't check, *your* code is what turns that failure into a crash, several lines later and much harder to trace back.
- **Always match every `malloc` with exactly one `free`.** Not zero (a leak). Not two (a double free). Exactly one, on every possible path through the function -- exactly the bug in this module's exercise.

- **Compile with `-Wall` and read every warning.** The compiler already catches some of these bugs for free, before you ever need a debugger -- an uninitialized variable used before being set is a common one it flags directly. A warning that's ignored today is a `gdb` session tomorrow.

Common beginner mistakes

- **Forgetting `-g`.** Running `gdb` on a program compiled without it gives you raw addresses instead of file names and line numbers -- technically debuggable, practically painful. If `gdb` can't find your source lines, this is almost always why.
- **Assuming "it didn't crash" means "it's correct."** A memory leak or an out-of-bounds read can run for a long time looking completely fine. Absence of a crash is not proof of absence of a bug -- that's exactly the gap `valgrind` fills.
- **Reading only the first line of a `valgrind` report.** The "definitely lost" or "Invalid read" summary tells you *that* something's wrong; the backtrace underneath tells you *where*. Skip it and you'll fix the wrong line, or nothing at all.
- **Freeing a pointer and continuing to use it.** `free()` releases the memory but doesn't change the pointer variable itself -- it still holds the old (now invalid) address. Set it to `NULL` right after freeing, as a habit, not just when you remember to.
- **Panicking at "Segmentation fault" with no other output.** That bare message is the *starting* point for debugging, not a dead end -- it's exactly what the hook showed you `gdb` turns into a precise answer in seconds.

Try it yourself

1. Work through every file in `examples/`, compiling each with `gcc -g -Wall` and running it. For 01, run it once plain, then again under `gdb` and reproduce the backtrace yourself. For 02 and 03, run them under `valgrind` if you have it; if not, read through the code and match it against the reconstructed reports above -- explain in your own words what each report line means before moving on.
2. Complete `exercises/exercise1.c` -- find and describe the bug yourself before checking `exercises/solutions/exercise1_solution.c`.
3. Build the **module project**: a program with three separate bugs, waiting for you to diagnose and fix all of them.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 15: debugging and memory safety"
git push
```

Next: **Module 16 — Enums, Unions & Bitwise Operators.**

Module 16 — Enums, Unions & Bitwise Operators

Hook: one number, eight switches

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>

#define FLAG_READ      (1 << 0)
#define FLAG_WRITE     (1 << 1)
#define FLAG_EXECUTE   (1 << 2)
#define FLAG_DELETE    (1 << 3)
#define FLAG_SHARE     (1 << 4)
#define FLAG_ADMIN     (1 << 5)
#define FLAG_BACKUP    (1 << 6)
#define FLAG_AUDIT     (1 << 7)

void printPermissions(int perms) {
    printf("READ=%d WRITE=%d EXECUTE=%d DELETE=%d SHARE=%d ADMIN=%d BACKUP=%d\n",
        (perms & FLAG_READ)      != 0,
        (perms & FLAG_WRITE)     != 0,
        (perms & FLAG_EXECUTE)   != 0,
        (perms & FLAG_DELETE)    != 0,
        (perms & FLAG_SHARE)     != 0,
        (perms & FLAG_ADMIN)     != 0,
        (perms & FLAG_BACKUP)    != 0,
        (perms & FLAG_AUDIT)     != 0);
}

int main(void) {
    int userPermissions = FLAG_READ | FLAG_WRITE | FLAG_EXECUTE;

    printf("Before:\n");
    printPermissions(userPermissions);

    userPermissions |= FLAG_SHARE;    // flip ONE switch on, nothing else

    printf("After granting SHARE:\n");
    printPermissions(userPermissions);

    return 0;
}

gcc hook.c -o hook
./hook
Before:
READ=1 WRITE=1 EXECUTE=1 DELETE=0 SHARE=0 ADMIN=0 BACKUP=0 AUDIT=0
After granting SHARE:
READ=1 WRITE=1 EXECUTE=1 DELETE=0 SHARE=1 ADMIN=0 BACKUP=0 AUDIT=0
```

One number, eight switches, and one line of code (`userPermissions |= FLAG_SHARE;`) flipped exactly one of them — every other flag, untouched. There's no array, no eight separate `bool` variables, just a single `int` holding eight independent yes/no answers at once. By the end of this module you'll know exactly how that line works, and you'll build a fuller version of this permissions system in this module's project.

enum: naming a fixed set of related constants

Back in Module 2 you met `int`, `float`, `double`, and `char`. An **enum** (short for "enumeration") is a type you define yourself: a fixed, named set of integer constants that all belong together.

```
enum GameState { MENU, PLAYING, PAUSED, GAME_OVER };
```

This declares four named constants — `MENU`, `PLAYING`, `PAUSED`, `GAME_OVER` — and a new type, `enum GameState`, that you can use just like `int` when declaring a variable:

```
enum GameState state = PLAYING;
```

Unless you say otherwise, C numbers enum members automatically starting at 0: `MENU` is 0, `PLAYING` is 1, `PAUSED` is 2, `GAME_OVER` is 3. You can also assign your own values explicitly:

```
enum Direction { NORTH = 1, SOUTH = 2, EAST = 4, WEST = 8 };
```

See `examples/01_enums_basics.c`.

Why bother? Replacing magic numbers

Before enums, code that tracks "what state is the game in?" often looks like this:

```
if (state == 1) {
    // ... playing
}
```

That 1 is a **magic number** — a value with meaning that exists only in the programmer's head (or a comment that will eventually go stale). Six months from now, is 1 "playing," or did someone renumber the states? With an enum, the same check reads as plain English and the compiler still sees the same underlying integer comparison:

```
if (state == PLAYING) {
    // ... playing
}
```

See `examples/02_enums_vs_magic_numbers.c` for the same logic written both ways, side by side.

Enums are really just ints — convenience and limitation

Under the hood, an enum value *is* an `int`. That's exactly why `printf("%d", state)` works with no special format specifier, and why you can compare an enum value with `==` just like any other integer. It's also the limitation: **C does not stop you from putting any `int` into an enum variable**, valid member or not.

```
enum GameState sneaky = 99; // compiles fine -- 99 isn't a real GameState
```

There's no real type safety here — an enum is a naming convenience for you and future readers, not a guarantee enforced by the compiler. Keep that in the back of your mind; it won't bite you often, but it can.

Bitwise operators: working on individual bits

Every value in your program is, underneath, a sequence of bits — 0s and 1s. The **bitwise operators** let you inspect and manipulate those bits directly, rather than treating a number only as "a quantity." C has six: `&`, `|`, `^`, `~`, `<<`, and `>>`.

Take `a = 12` (binary 00001100) and `b = 10` (binary 00001010):

Operator	Name	Result on a, b	Rule
<code>a & b</code>	AND	00001000 (8)	1 only where both bits are 1
<code>a b</code>	OR	00001110 (14)	1 where either bit is 1
<code>a ^ b</code>	XOR	00000110 (6)	1 where the bits differ
<code>~a</code>	NOT	11110011 (243 as unsigned char)	every bit flipped
<code>a << 2</code>	left shift	00110000 (48)	bits move left, zeros fill in ($\approx a * 4$)
<code>a >> 2</code>	right shift	00000011 (3)	bits move right ($\approx a / 4$)

Run `examples/03_bitwise_operators.c` — it prints each of these out bit by bit so you can watch the patterns rather than just trust the table.

Do not confuse `&` and `|` with `&&` and `||`. `&&` and `||` are the *logical* operators from Module 2 — they work on whole true/false values and short-circuit. `&` and `|` are *bitwise* — they work bit by bit on the binary representation of a number. `6 && 1` is 1 (both nonzero, so both "true"). `6 & 1` is 0 (0110 and 0001 share no 1 bits). They can produce completely different answers on the same operands — mixing them up is a real, easy-to-make bug.

Flags: one number, several independent switches

The hook's real trick is combining `<<` with `|`: define each flag as a power of two — 1, 2, 4, 8, ... — so each one claims exactly one bit that none of the others use.

```
#define FLAG_READ    (1 << 0)    // 1    -- 00000001
#define FLAG_WRITE   (1 << 1)    // 2    -- 00000010
#define FLAG_EXECUTE (1 << 2)    // 4    -- 00000100
```

`1 << n` is a common, readable way to spell "the bit at position `n`" — it reads as "1, shifted left `n` places," which is clearer at a glance than memorizing that `FLAG_EXECUTE` happens to be 4.

With the bits laid out like that, one `int` variable can track many independent yes/no answers at once, using three small patterns:

```
int permissions = 0;

permissions |= FLAG_READ;           // SET a flag: OR it in
permissions |= FLAG_EXECUTE;

if (permissions & FLAG_READ) {      // CHECK a flag: AND with it, see if
    anything survives
    printf("READ is set.\n");
}

permissions &= ~FLAG_READ;          // CLEAR a flag: AND with everything EXCEPT
that bit
```

`~FLAG_READ` flips every bit of `FLAG_READ`, so ANDing with it turns *only* that one bit off and leaves every other bit exactly as it was — which is exactly what the hook demonstrated: granting `FLAG_SHARE` touched nothing else. See [examples/04_flags_with_bitwise_or_and_and.c](#).

union: different types sharing the same memory

A **union** looks like a `struct`, but behaves very differently: all of its members share the *same* block of memory instead of each getting their own. Only one member is meaningful at a time — writing through one member and then reading through a *different* member does not convert the value, it **reinterprets the same raw bytes** as a different type, which is almost always a bug.

```
union {
    int i;
    float f;
} v;

v.i = 42;
printf("%f\n", v.f); // NOT 42.0 -- same bits, read as a float instead
```

Because that's dangerous on its own, real code almost always pairs a union with a separate **tag** field that records which member is currently valid — check the tag before you read:

```
enum ValueType { TYPE_INT, TYPE_FLOAT };

typedef struct {
    enum ValueType type; // the tag
    union {
        int i;
        float f;
    } data;
} Variant;
```

Now `if (v.type == TYPE_INT) { ... v.data.i ... }` is always reading the member that was actually written. This tagged-union pattern is exactly how safe, real-world code uses unions. See [examples/05_unions_basics.c](#), which shows both the safe tagged version and the unsafe "read the wrong member" bug, side by side.

Scope check: what you'll actually reach for

Of the three tools in this module, you'll use them at very different rates. **Enums** are something you'll reach for often — anywhere you'd otherwise scatter magic numbers, an enum is usually the right call. **Unions** and **raw bit manipulation** are more specialized: you might write only a handful of unions in

typical beginner/intermediate code, and most day-to-day flag-like problems in C are solved with `bool` variables or arrays long before you'd reach for hand-rolled bitwise flags. That said, recognizing both when you meet them — in a library header, a device driver, a file format, or a job's existing codebase — matters, and now you can.

Common beginner mistakes

- Confusing `&|` (bitwise) with `&&||` (logical) — they can produce different results on the same values, not just different syntax.
- Assuming an enum variable can only ever hold a "real" member value — C won't stop you from assigning any `int`.
- Forgetting that `~FLAG_X` is required to clear a flag — `permissions &= FLAG_X`; (no `~`) does the opposite of what you want.
- Writing one union member and reading a different one, expecting a conversion — you get the same bytes reinterpreted, not a converted value.
- Choosing flag values that aren't distinct powers of two (e.g. `1`, `2`, `3`) — `3` overlaps with both `1` and `2`'s bits, breaking the one-bit-per-flag guarantee.
- Forgetting that shifting and masking bind more loosely than you might expect in a larger expression — when unsure, add parentheses, exactly as Module 2 taught you for arithmetic.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one. For `03_bitwise_operators.c`, predict each binary result before you run it, then check.
2. Complete **exercises/exercise1.c**.
3. Build the **module project**.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 16: enums, unions, and bitwise operators"
git push
```

Next: **Module 17 — Command-Line Arguments & CLI Tools.**

Module 17 — Command-Line Arguments & CLI Tools

Hook: one program, two completely different behaviors

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s <name> [--shout]\n", argv[0]);
        return 1;
    }

    char *name = argv[1];
    int shout = 0;

    for (int i = 2; i < argc; i++) {
        if (strcmp(argv[i], "--shout") == 0) {
            shout = 1;
        }
    }

    if (shout) {
        printf("HELLO, ");
        for (int i = 0; name[i] != '\0'; i++) {
            putchar(toupper(name[i]));
        }
        printf("!!!\n");
    } else {
        printf("Hello, %s!\n", name);
    }

    return 0;
}

gcc hook.c -o greet
./greet Ama
./greet Kwame --shout
```

`./greet Ama` prints `Hello, Ama!`. `./greet Kwame --shout` prints `HELLO, KWAME!!!` — same compiled program, zero code changes, entirely different behavior. The difference wasn't a new build; it was information handed to the program **from outside itself**, at the moment it started. By the end of this module you'll understand exactly how `main` receives that information, and you'll build a small `wc`-style tool of your own in the module project.

A new shape of `main`

Every program you've written until now has started with:

```
int main(void)
```

`void` means "takes no inputs." There's a second, equally standard form:

```
int main(int argc, char *argv[])
```

This `main` *does* take inputs — the words typed after the program's name on the command line. `argc` ("argument count") is a whole number telling you how many of those words there are. `argv` ("argument vector") is an array of C-strings, one per word. Both are filled in automatically by the operating system before your program's first line even runs — you don't request them, they're just there.

`argv[0]` is the program's own name

This trips up nearly everyone the first time: `argv[0]` is **not** the first argument the user typed — it's the name (or path) the program was run as. The first *real* argument is `argv[1]`.

Run `./greet Ama` and the array looks like this:

```
argc = 2
argv[0] = "./greet"    <- the program's own name, always
argv[1] = "Ama"        <- the first thing the user actually typed
```

Run `./greet Kwame --shout` (three words, including the program name) and `argc` becomes 3, with `argv[2] = "--shout"`. See [examples/01_argc_and_argv_basics.c](#) — run it with a few different sets of arguments and watch `argc` and every slot of `argv` print out.

Always check `argc` before indexing `argv`

`argv` is an array, and Module 5's rule about arrays never went away: reading an index that wasn't actually filled in reads **past the end of the array** — undefined behavior, not a clean error message. If a program blindly reads `argv[1]` and the user ran it with zero arguments, `argv[1]` doesn't exist.

```
// DANGEROUS: assumes argv[1] exists
printf("Hello, %s!\n", argv[1]);

// SAFE: check argc first
if (argc < 2) {
    printf("Usage: %s <name>\n", argv[0]);
    return 1;
}
printf("Hello, %s!\n", argv[1]);
```

Every example and exercise in this module checks `argc` before it touches `argv[1]` or beyond — get in the habit now.

Reading named flags by hand

A **flag** is an argument that turns a behavior on or off, usually starting with `--` (e.g. `--shout`). There's no special language feature for this — you just loop over `argv` and compare each entry against the flag strings you're expecting, using `strcmp` (Module 5):

```
int shout = 0;
for (int i = 2; i < argc; i++) {
    if (strcmp(argv[i], "--shout") == 0) {
        shout = 1;
    }
}
```

This is exactly what the hook's `greet` program does — see [examples/02_reading_named_flags.c](#) for the full version. It's worth knowing that real-world

C programs usually parse flags like this with the standard library's `getopt()` / `getopt_long()` instead of a hand-rolled loop — it handles things like `-x` vs `--long-name` and combined short flags for you. It's not necessary for anything in this module, but worth looking up once `strcmp`-based parsing feels comfortable.

Arguments are always strings — converting to numbers

No matter what the user types, `argv[i]` is a C-string. Typing `./program 42` does **not** hand your program the number 42 — it hands it the two characters `'4'` and `'2'`. To actually do arithmetic with it, convert it first:

```
int count = atoi(argv[1]); // "42" -> 42
```

`atoi` ("ASCII to integer") is the quick tool for this. Its downside: given text that isn't a valid number at all (like `"abc"`), it quietly returns 0 — it can't tell you "that wasn't a number" versus "that number really was 0." For real error-checking, `strtol` is the more robust tool (it can report exactly where parsing failed), though its full error-handling interface is more than this module needs — `atoi` is enough for now. See `examples/04_combining_flags_and_a_filename_argument.c` and `examples/05_printing_usage_help.c`.

The shape of a real Unix-style CLI tool

Every program in this course so far has followed the same interactive loop: print a menu, `scanf` a choice, act on it, repeat until the user picks "quit." That's a perfectly reasonable shape for a program a human sits in front of — but it's not how most real command-line tools work. Commands like `wc`, `grep`, and `ls` follow a different, simpler shape:

```
read input (from arguments and/or a file) -> do ONE job -> print output ->
exit
```

No menu. No "run again? (y/n)." The tool is meant to be driven **from outside itself** — by a human typing different arguments each time, by a script that calls it in a pipeline, or by another program entirely. Everything the tool needs to know is handed to it as arguments (and maybe a file) when it starts, and it's gone the moment it finishes. `examples/03_a_real_cli_tool_word_counter.c` reads a file named on the command line, counts its words, prints one number, and exits — the whole thing in one pass, no interaction at all.

Printing a usage message

When a CLI tool is called wrong — missing arguments, a bad flag — the helpful response isn't a crash or silent wrong output. It's a short **usage message** showing exactly how the program expects to be called, the CLI equivalent of a helpful compiler error:

```
if (argc != 3) {
    printf("Usage: %s <number1> <number2>\n", argv[0]);
    return 1;
}
```

Using `argv[0]` (rather than hardcoding the program's name) means the message is always accurate even if the compiled file gets renamed. Returning a non-0 value (Module 1) also signals to whatever *called* your program — a script, another tool — that something went wrong, which matters a great deal once programs start calling other programs.

Common beginner mistakes

- Assuming `argv[1]` is the first argument the user typed — it's `argv[0]` that's the program's own name; `argv[1]` is the first real one.
- Reading `argv[i]` without checking `argc` first — an out-of-bounds array read, not a clean error.
- Forgetting that every `argv[i]` is a **string**, even when it looks like a number — `argv[1] + argv[2]` doesn't add two numbers, it adds two pointers. Convert with `atoi/strtol` first.
- Writing `if (argv[i] == "--shout")` instead of `if (strcmp(argv[i], "--shout") == 0)` — Module 5's rule about comparing C-strings with `strcmp`, not `==`, still applies here.
- Consuming a flag's value argument (like the `N` after `--lines`) without checking there's actually another argument left to read.
- Building an interactive menu loop when the assignment asked for a CLI tool — re-read the requirements; "do one job and exit" is the point.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one **with real command-line arguments** — these programs don't do anything interesting run with no arguments at all (except printing their usage message).
2. Complete **exercises/exercise1.c**.
3. Build the **module project**.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 17: command-line arguments and CLI tools"
git push
```

Next: **Module 18 — Sorting & Searching Algorithms.**

Module 18 — Sorting & Searching Algorithms

Hook: the same search, run two ways

Compile and run this exactly as written — don't read ahead first:

```

#include <stdio.h>

#define SIZE 1000

int linearSearch(int arr[], int size, int target, int *comparisons) {
    for (int i = 0; i < size; i++) {
        (*comparisons)++;
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}

int binarySearch(int arr[], int size, int target, int *comparisons) {
    int low = 0;
    int high = size - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        (*comparisons)++;
        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main(void) {
    static int arr[SIZE];
    for (int i = 0; i < SIZE; i++) {
        arr[i] = i * 2;    // sorted: 0, 2, 4, ..., 1998
    }

    int target = 1998;    // the very last element -- linear search's worst
case

    int linearComparisons = 0;
    int linearResult = linearSearch(arr, SIZE, target, &linearComparisons);

    int binaryComparisons = 0;
    int binaryResult = binarySearch(arr, SIZE, target, &binaryComparisons);

    printf("Searching for %d among %d sorted numbers...\n\n", target, SIZE);
    printf("Linear search: found at index %d, took %d comparisons\n",
linearResult, linearComparisons);
    printf("Binary search: found at index %d, took %d comparisons\n",
binaryResult, binaryComparisons);

    return 0;
}
gcc hook.c -o hook
./hook

```

```
Searching for 1998 among 1000 sorted numbers...
```

```
Linear search: found at index 999, took 1000 comparisons
```

```
Binary search: found at index 999, took 10 comparisons
```

Same array, same target, same answer — **1000 comparisons versus 10**. Both functions are correct. Both find the right answer. One of them just does dramatically less work, because it's allowed to assume something about the array that the other isn't. By the end of this module you'll know exactly what that assumption is, how to write both kinds of search, how to write the sorting algorithms that make binary search possible in the first place, and you'll build a small leaderboard tool in this module's project.

Linear search: check everything

Linear search is the obvious approach: start at the beginning, look at each element in order, and stop the moment you find the target (or run out of array). That's it — no requirements, no setup.

```
int linearSearch(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            return i;    // found it -- return the index immediately
        }
    }
    return -1;          // walked the whole array, never matched
}
```

The one genuine advantage of linear search: **it works on any array, sorted or not**. The cost: in the worst case (the target is the very last element, or isn't there at all), it has to check every single element. For an array of 1,000 elements, that's up to 1,000 comparisons; for a million elements, up to a million. The bigger the array, the more work linear search might have to do — it grows right along with the data. See [examples/01_linear_search.c](#).

Binary search: throw away half the possibilities every step

Binary search is far faster, but it has a precondition: **the array must already be sorted**. Given that, it doesn't need to check every element — it can compare the target to the *middle* element and immediately know which half of the array to ignore entirely.

```

int binarySearch(int arr[], int size, int target) {
    int low = 0;
    int high = size - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            return mid;           // found it
        } else if (arr[mid] < target) {
            low = mid + 1;         // target must be in the right half
        } else {
            high = mid - 1;        // target must be in the left half
        }
    }

    return -1; // low crossed high -- target isn't in the array
}

```

Trace it by hand: searching a sorted 15-element array {2, 5, 8, 12, 16, 23, 38, 42, 45, 51, 59, 63, 71, 78, 90} for 71:

```

low=0  high=14 mid=7  arr[7]=42  42 < 71  -> search the right half
low=8  high=14 mid=11 arr[11]=63  63 < 71  -> search the right half
low=12 high=14 mid=13 arr[13]=78  78 > 71  -> search the left half
low=12 high=12 mid=12 arr[12]=71  found!

```

Four comparisons to find one value out of fifteen — and every single step throws away **half** of whatever was still left, so the advantage only gets more extreme as the array grows. That's exactly what the hook demonstrated: 1,000 elements, and binary search still only needed 10 comparisons. See [examples/02_binary_search.c](#), which prints this exact trace as it runs.

The precondition is not optional

Here's the part that matters most: **binary search on an unsorted array does not crash, does not error, does not warn you.** It runs, it produces *an* index, and that index is sometimes just wrong — because the whole algorithm's logic ("the target isn't here, so it must be in this half") only holds up when the array is actually sorted. On an unsorted array, that assumption breaks silently, and you get a plausible-looking wrong answer instead of a message telling you something went wrong. A crash is annoying; a silent wrong answer is *much* scarier, because nothing tells you to go looking for the bug. **Always know whether the array you're searching is actually sorted before reaching for binary search** — if it isn't, either sort it first (see below) or fall back to linear search.

Sorting: three ways to put an array in order

Binary search needs a sorted array to work — so before you can use it, something has to *produce* that sorted array. That's what the rest of this module covers: three classic sorting algorithms that all do the same job (rearrange an array into ascending order) with different strategies.

Bubble sort

Repeatedly walk the array comparing **adjacent pairs**, swapping them if they're out of order. Each full pass pushes the largest remaining value one step closer to the end — like a bubble rising to the top.

```

void bubbleSort(int arr[], int size) {
    for (int pass = 0; pass < size - 1; pass++) {
        for (int i = 0; i < size - 1 - pass; i++) {
            if (arr[i] > arr[i + 1]) {
                int temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
            }
        }
    }
}

```

Bubble sort is the simplest sort to understand and hand-trace, which is exactly why it's a good *first* sort to learn — but it's also one of the slowest for large arrays, doing far more comparisons and swaps than it needs to. Learn it for the intuition, then reach for something else (insertion sort, or the standard library's `qsort`) on real data. See `examples/03_bubble_sort.c`.

Selection sort

For each position, starting at index 0, find the **minimum** value in the remaining unsorted part of the array and swap it into that position.

```

void selectionSort(int arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < size; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
}

```

After pass `i`, the first `i + 1` elements are the `i + 1` smallest values in the array, fully sorted, and every later pass only ever touches the unsorted remainder. See `examples/04_selection_sort.c`.

Insertion sort

Build up a sorted portion at the *front* of the array, one element at a time — exactly like sorting a hand of playing cards as you pick them up. Each new element gets shifted left, past every already-sorted card bigger than it, until it lands in its correct spot.

```

void insertionSort(int arr[], int size) {
    for (int i = 1; i < size; i++) {
        int key = arr[i];          // the card we just "picked up"
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];    // shift a bigger sorted element right
            j--;
        }
        arr[j + 1] = key;          // drop key into its correct spot
    }
}

```

See `examples/05_insertion_sort.c`.

A first sense of "how much work is that, really?"

You don't need formal algorithm analysis to notice that these algorithms don't all cost the same. Just count, roughly, how many comparisons each one might need in the worst case, for a few different array sizes:

Array size	Linear search / bubble / selection / insertion sort (roughly)	Binary search (roughly)
10	up to 10 (search) / ~100 (sort)	~4
1,000	up to 1,000 / ~1,000,000	~10
1,000,000	up to 1,000,000 / ~1,000,000,000,000	~20

Two things jump out:

- **Linear search and all three sorts above do more and more work as the list grows** — the sorts especially, since comparing *every pair* against *every other* element scales up fast (this is why they're sometimes grouped as "quadratic" sorts).
- **Binary search barely grows at all**, even for a huge list — going from 1,000 to 1,000,000 elements (1000x bigger) only costs it about 10 more comparisons, because it throws away half of whatever's left at every single step.

That's the whole intuition. (There's a formal way to describe this precisely — "Big-O notation" — but that's a topic for a later course; for now, the important habit is just noticing that different algorithms can do wildly different amounts of work for the same job.)

A quick, honest look at `qsort`

Every algorithm above is worth learning for the intuition — but real production C code almost never hand-writes its own sort. The standard library already provides one: `qsort`, from `<stdlib.h>`.

```
int compareInts(const void *a, const void *b) {
    return (*(int *)a) - (*(int *)b);
}
```

```
qsort(values, size, sizeof(int), compareInts);
```

`qsort` takes the array, how many elements it has, the size of one element, and a **comparator function** you write — a small function that takes two generic pointers, casts them back to the real type, and returns negative/zero/positive depending on which value should come first. You don't need to understand function pointers deeply to use this (that's its own topic, for later) — just recognize the pattern: write a comparator with exactly this shape, hand it to `qsort`, and it does the sorting. See **examples/06_qsort_from_the_standard_library.c** for a complete, working example.

Common beginner mistakes

- Running binary search on an array that **isn't actually sorted** — it won't crash, it'll just sometimes return the wrong answer, with no indication anything went wrong. Always verify the array is sorted first.
- Off-by-one errors in binary search's bounds — using `low < high` instead of `low <= high` (misses the case where the target is exactly at `low == high`), or forgetting `mid + 1 / mid - 1` and looping forever because `low/high` never actually move.
- Writing `mid = (low + high) / 2` instead of `mid = low + (high - low) / 2` — rarely matters for small arrays in this course, but the second form avoids integer overflow on very large arrays and is worth building as a habit.
- Forgetting that a sort function needs to modify the array **in place** (via the array parameter) — the caller's array only changes if the sort actually swaps elements inside it, not a local copy.
- In bubble/selection/insertion sort, mixing up `<` and `<=`, or `i` and `j`, in the inner loop bounds — a one-character typo silently sorts the array *almost* correctly, which is a genuinely hard bug to spot by eye. Always test a small array by hand, not just by staring at the code.
- Using `==` instead of `strcmp` if you experiment with sorting strings later — a reminder all the way back from Module 5, and it still applies here.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one.
2. Complete **exercises/exercise1.c**.
3. Build the **module project**.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 18: sorting and searching algorithms"
git push
```

Next: **Module 19 — Part 2 Capstone Project**.

Module 19 — Part 2 Capstone Project

Part 1's capstone proved you could combine variables, control flow, arrays, functions, pointers, structs, and file I/O into one real program. This capstone raises the bar: your data now has to **grow**, your program has to be **organized across files**, and at least one part of it has to **recurse** — the actual dividing line between a beginner and an intermediate C programmer.

1. Two ways to start

You have a genuine choice for this capstone:

- **Option A — Level up your Part 1 capstone.** Take the project you (or your team) built for **Module 9** and rebuild its core data storage as a linked list instead of a fixed-size array, split the code across `.h/ .c` files, and add a real recursive function somewhere it belongs. This is a great option if you're proud of what you already built and want to make it genuinely better rather than start over.
- **Option B — Pick a new project** from the menu in §4 below.

Either way, the requirements in §3 apply in full.

2. Teams and GitHub Classroom

Team formation works exactly like Module 9: your instructor announces the class size, which sets pairs (2:1, ≤ 20 students), groups of four (4:1, > 20 students), or an approved solo "daring" track (1:1 — same core requirements, plus every stretch goal). If your class is continuing straight from Part 1, you may keep the same team, or your instructor may have you re-form them — check before Sprint 0. Team formation and repo setup happen through a GitHub Classroom Group Assignment exactly as described in **Module 9 §2**.

3. Requirements for every project (Option A or B)

- **A dynamically-growing data structure**, built with `malloc/free` (Module 11) — a linked list, stack, or queue (Modules 12–13). A fixed-size array standing in for "the list" does not satisfy this.
- **No memory leaks.** Every `malloc` needs exactly one matching `free`. Document in your README how you verified this — `valgrind` if your platform supports it (Module 15), or a careful manual audit of every allocation if not.
- **At least one genuinely recursive function**, used because the problem calls for it — not bolted on to satisfy this checklist.
- **Code organized across multiple `.c/ .h` files** with include guards (Module 14) — not one giant `main.c`.
- **Everything Part 1 already required still applies:** structs, file I/O so your data survives a restart, functions with clear jobs, pointers used with intention, a clean `gcc -Wall` build, and a project README a stranger could build and run unaided.
- A menu-driven **or** command-line-argument-driven interface (Module 17) — pick whichever fits your project better, and say which you chose and why in your README.

4. Project menu (if you're not leveling up your Part 1 project)

1. **Task Manager** — a linked list of dynamically allocated tasks: add, complete, delete, reorder, saved to a file. *Stretch*: sort by priority (Module 18), an undo stack (Module 13) for the last change.
2. **Library System v2** — Module 9's library idea, rebuilt on a linked list catalog instead of a fixed array, so it can hold any number of books. *Stretch*: binary search (Module 18) over a sorted snapshot.
3. **Music Playlist Manager** — a linked list of songs: add, remove, reorder, shuffle, save/load. *Stretch*: an undo stack for the last edit.
4. **Undo/Redo Simulator** — two stacks (undo and redo) of dynamically allocated action records, simulating a text editor's edit history. *Stretch*: persist the history to a file.
5. **Command-Line Notes Tool** — a real CLI utility (Module 17, `argc/argv`-driven, no interactive menu) that appends, lists, and deletes notes stored in a linked list synced to a file, organized across multiple files. *Stretch*: search notes by keyword.
6. **Maze Solver** — recursive backtracking (Module 10) through a small grid maze, finding and printing a path. *Stretch*: load the maze from a file.
7. **Polynomial Calculator** — represent a polynomial as a linked list of `(coefficient, exponent)` nodes; add two polynomials together and print the result. *Stretch*: multiply two polynomials.

Have your own idea? Propose it to your instructor — as long as it satisfies every requirement in §3, it's fair game.

5. Suggested timeline

Sprint	Focus	Git evidence expected
Sprint 0	Team formed, project chosen, the dynamic data structure designed on paper, header file layout planned	An initial commit with your struct/node definitions and a <code>.h</code> file skeleton
Sprint 1	The dynamic structure works in memory (create/insert/delete/traverse), no persistence yet	Several small commits, each compiling cleanly
Sprint 2	File I/O persistence, and the code actually split across <code>.c/.h</code> files	A commit where you can close and reopen the program without losing data
Sprint 3	Recursion integrated meaningfully, interface polish, edge cases	Commits addressing specific bugs/edge cases you found
Final	Leak-free (documented), <code>-Wall</code> -clean build, finished README, last push before deadline	Your final commit is your submission

6. How this gets assessed

- **Correctness** — does it do what it claims, including edge cases (an empty list, a full free-and-reload cycle, bad input)?
- **Memory discipline** — every `malloc` freed, no leaks, no use-after-free.
- **Meaningful use of recursion, linked structures, and multi-file organization** — used appropriately, not bolted on.
- **Code organization** — logically split across functions and files, consistent style, comments where the *why* isn't obvious.
- **Git history and documentation** — regular, meaningful commits from every team member under their own name, and a README a stranger could build and run unaided.

Try it yourself

1. Confirm your team (or solo status) in GitHub Classroom.
2. Decide: level up your Part 1 project, or pick a new one from §4.
3. Design your core data structure on paper before writing code.
4. Work through the sprints in §5, committing and pushing regularly.
5. Submit by pushing your final commit before the deadline.

You've now built two real, working programs — one straightforward, one that manages its own memory correctly. That second one is the actual skill that separates "I can write C" from "I've written some C once."